

Introduction

DISTRIBUTED AND PARALLEL TRAINING



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

1

1

Huấn luyện song song phân tán

Teams code: amngynu



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

Distributed machine learning system

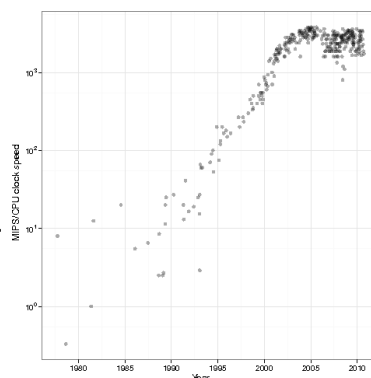


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

3

Motivation for Parallel and Distributed Systems

- Clock speeds aren't increasing anymore:
 - Though new tricks like 64-bit vs. 32-bit.
- But datasets keep getting bigger.
 - MNIST: 60k, ImageNet: 1.4M.
- We need to use parallel computation.
 - Use more than 1 CPU to reduce time.
 - Lets you keep pace with growth of data.



<https://csgillespie.wordpress.com/2011/01/25/cpu-and-gpu-trends-over-time/>



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

4

Motivation for Parallel and Distributed Systems

- Data might get so big it doesn't fit on one machine.



- We need to consider **distributed data** and **distributed computation**.
 - How can we solve ML problems efficiently in this setting?

https://en.wikipedia.org/wiki/Hard_disk_drive



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

5

3 Approaches to Machine Learning

- There are roughly three computational approaches to ML:
 - **Counting** (sufficient statistics, decision trees, naïve Bayes, KNN).
 - **Optimization** (least squares, logistic regression, PCA, deep learning).
 - **Integration** (random forests, graphical models, Bayesian methods).
- **Issues** arising in these settings when you parallelize/distributed.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

6

An ML Program

$$\arg \max_{\mathcal{M}} L(\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N ; \mathcal{M}) + \lambda \mathcal{R}(\mathcal{M})$$

Model Data Parameter

Solved by an iterative convergent algorithm

```
for (t = 1 to T) {
  doThings()
   $\mathcal{M}^{t+1} = g(\mathcal{M}^t, f(\mathcal{M}^t(D)))$ 
  doOtherThings()
}
```



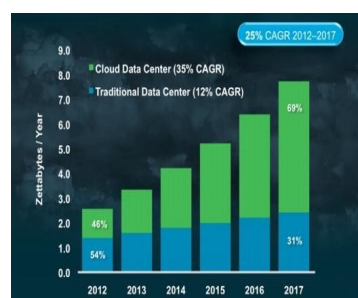
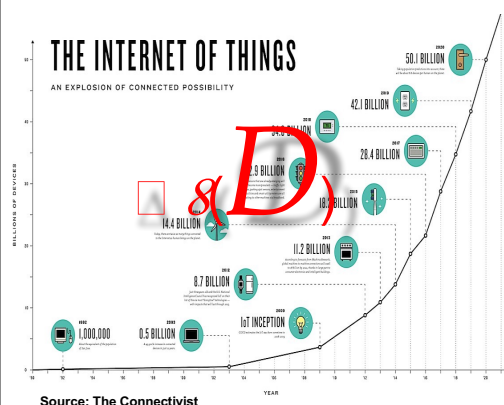
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

This computation needs to be scaled up !

7

7

Challenge 1 – Massive Data Scale



Source: Cisco Global Cloud Index

Familiar problem: data from 50B devices, data centers won't fit into memory of single machine

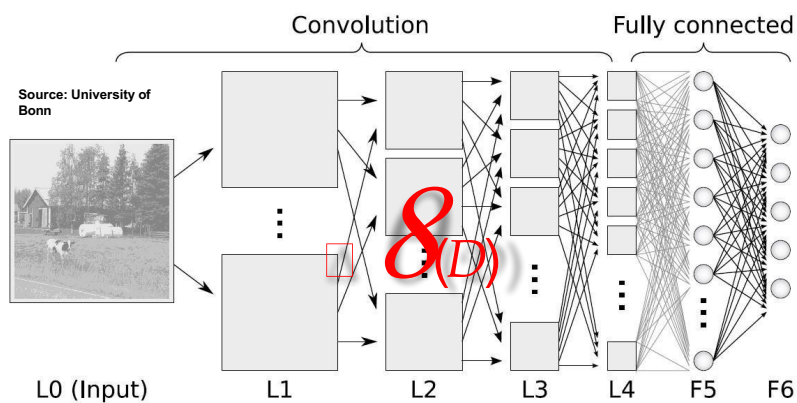


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

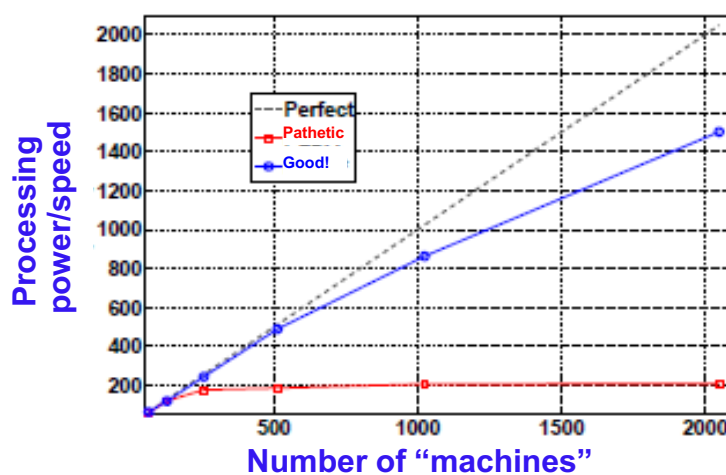
8

Challenge 2 – Gigantic Model Size

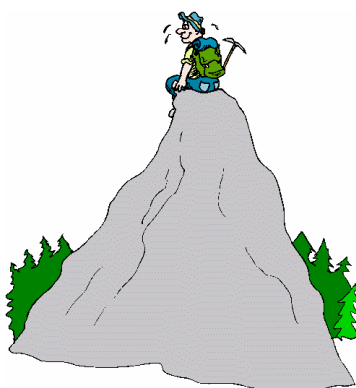


Maybe Big Data needs Big Models to extract understanding?
But models with >1 trillion params also won't fit!

The Scalability Challenge



ML Computation vs. Classical Computing Programs



ML Program:
optimization-centric and
iterative convergent



Traditional Program:
operation-centric and
deterministic



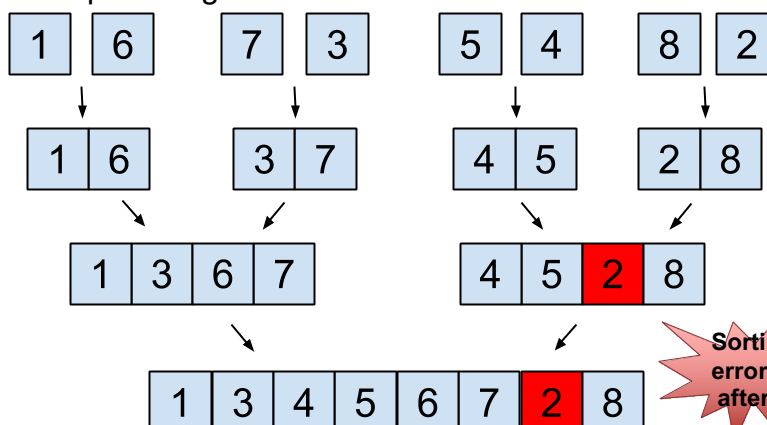
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

11

Traditional Data Processing needs operational correctness ...

Example: Merge sort



Sorting
error: 2
after 5



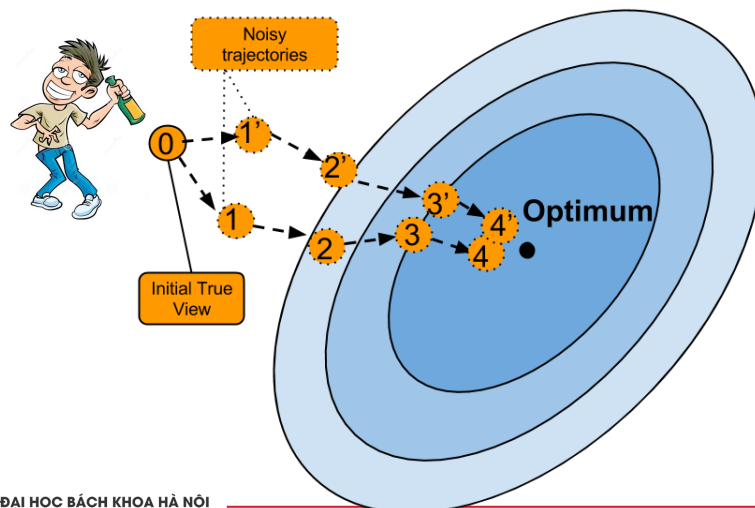
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Error persists and is
not corrected

12

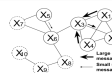
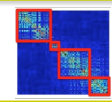
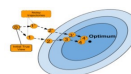
12

... but ML Algorithms can Self-heal



More Intrinsic Properties of ML Programs

- ML is **optimization-centric**, and admits an **iterative convergent** algorithmic solution rather than a one-step closed form solution
- Error tolerance**: often robust against limited errors in intermediate calculations
- Dynamic structural dependency**:
changing correlations between model parameters
critical to efficient parallelization
- Non-uniform convergence**: parameters can converge in very different number of steps
- Whereas traditional programs are **transaction-centric**, thus only guaranteed by **atomic correctness** at every step



An ML Program

$$\arg \max_{\mathcal{V}} L(\{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^N; \mathcal{V}) + \lambda \|\mathcal{V}\|$$



Solved by an iterative convergent algorithm

```
for (t = 1 to T) {
  doThings()
   $\mathcal{V}^{t+1} = g(\mathcal{V}^t, f(\mathcal{V}^t(D)))$ 
  doOtherThings()
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

This computation needs to be parallelized!

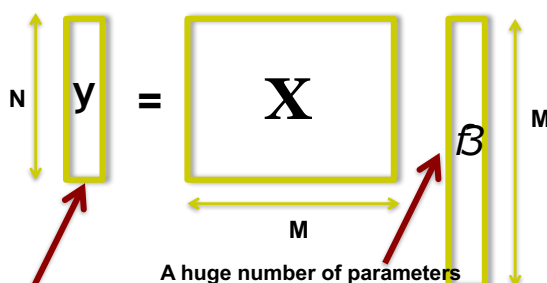
15

15

Challenge

- Optimization programs:

$$\sum_{i=1}^N \frac{d}{d\mathcal{V}_1} \dots \frac{d}{d\mathcal{V}_M} f(\mathbf{x}_i, \mathbf{y}_i; \mathcal{V})$$



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

A huge volume of data (e.g.) $N = 1B$
A huge number of parameters (e.g.) $M = 1B$

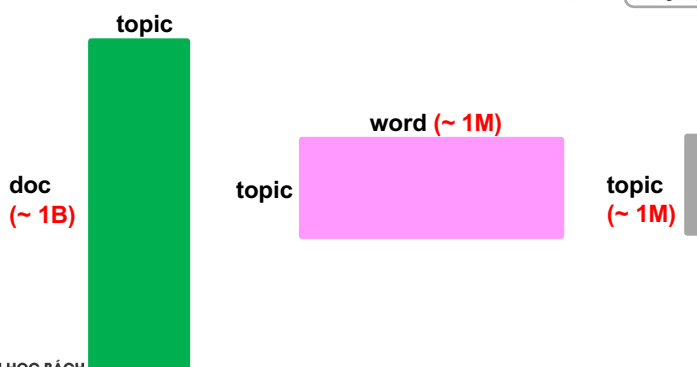
16

16

Challenge

- Probabilistic programs

$$z_{ij} \sim p(z_{ij} = k | x_{ij}, \delta_i, B) \propto (\delta_{ik} + \alpha_k) \cdot \frac{\beta_{x_{ij}} + B_{k,x_{ij}}}{V\beta + \sum_{v=1}^V B_{k,v}}$$

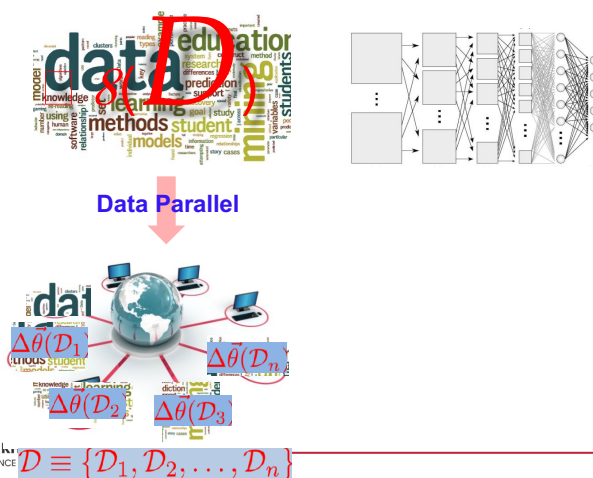


17

Parallelization Strategies

$$\vec{\theta}^{t+1} = \vec{\theta}^t + \Delta_f \vec{\theta}(\mathcal{D})$$

New Model = Old Model +
Update(Data)



$$\mathcal{D} \equiv \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_n\}$$

18

Parallelization Strategies

$$\vec{\theta}^{t+1} = \vec{\theta}^t + \Delta_f \vec{\theta}(\mathcal{D})$$

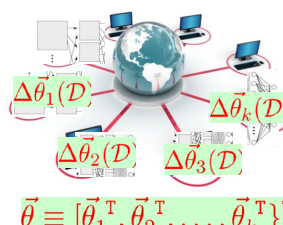
New Model = Old Model + Update(Data)



Data Parallel



Model Parallel



19

19

Distributed Algorithms

- Many parallel algorithms for Optimization, MCMC
- Common parallelization themes
 - **Embarrassingly parallel:** combine results from multiple independent problems, e.g. PSGD, EP-MCMC
 - **Stochastic over data:** approximate functions/ gradients with expectation over subset of data, then parallelize over data subsets, e.g. SGD
 - **Model-parallel:** parallelize over model variables, e.g. Coordinate Descent
 - **Auxiliary variables:** decompose problem by decoupling dependent variables, e.g. ADMM, Auxiliary Variable MCMC
- Considerations
 - **Regularizers, model structure:** may need sequential proximal or projection step, e.g. Stochastic Proximal Gradient
 - **Data partitioning:** for data-parallel, how to split data over machines?
 - **Model partitioning:** for model-parallel, how to split model over machines? **Need to be careful as model variables are not necessarily independent of each other.**



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

20

20

Implementing Distributed ML Algorithms

- Distributed ML requires care
- If not careful, slower than single machine!
 - Non-trivial systems bottlenecks (load imbalance, network bandwidth & latency)
- Even if algorithm is theoretically sound and has attractive properties, still need to pay attention **to system aspects**
 - Bandwidth (comms volume limits)
 - Latency (comms timing limits)
 - Data and Model partitioning (machine memory limitation, also affects comms volume)
 - Data and Model scheduling (affects convergence rate, comms volume & timing)
 - **Non-ideal systems behavior:** uneven machine performance, other cluster users

21

Implementing Distributed ML Algorithms

- Ad-hoc and partial solutions can lack theoretical analysis
 - **Major barrier:** hard to analyze solutions because algorithm/systems sometimes not fully/transparently described in papers
 - **Possible solution:** a universal language and principles for design could facilitate theoretical analysis of existing and new solutions
- Let us look at some software, which distributed ML algorithms can be implemented upon

22

Software to Implement Distributed ML

23

History of Systems for Big ML

- Data-, model-parallel ML algorithms for optimization, MCMC
- One could write distributed implementations from scratch
- Perhaps better to use an existing open source platform?



24

Spark Overview [Zaharia et al., 2010]



- General-purpose system for Big Data processing
 - Shell/interpreter for Matlab/R-like analytics
- MLlib = Spark's ready-to-run ML library
 - Implemented on Spark's API



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

25

25

Spark Overview [Zaharia et al., 2010]



- MLlib algorithms (v1.4)
 - Classification and regression
 - linear models (SVMs, logistic regression, linear regression)
 - naive Bayes
 - decision trees
 - ensembles of trees (Random Forests and Gradient-Boosted Trees)
 - isotonic regression
 - Collaborative filtering
 - alternating least squares (ALS)
 - Clustering
 - k-means
 - Gaussian mixture
 - power iteration clustering (PIC)
 - latent Dirichlet allocation (LDA)
 - streaming k-means
 - Dimensionality reduction
 - singular value decomposition (SVD)
 - principal component analysis (PCA)

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

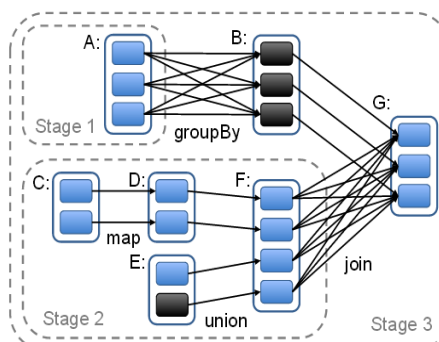
26

26

Spark Overview [Zaharia et al., 2010]



- Key feature: Resilient Distributed Datasets (RDDs)
 - Data processing = lineage graph of transforms
 - RDDs = nodes
 - Transforms = edges



Spark Overview [Zaharia et al., 2010]



- RDD-based programming model
 - Similar in spirit to Hadoop Mapreduce
 - Functional style: manipulate RDDs via "transformations", "actions"
 - E.g. map is a transformation, reduce is an action
 - Example: load file, count total number of characters

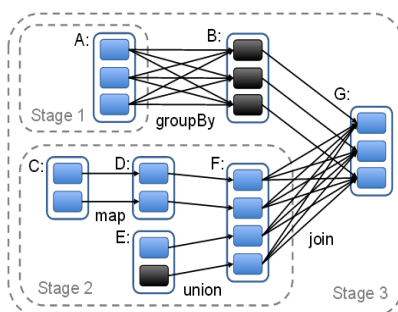
```
val lines = sc.textFile("data.txt")
val lineLengths = lines.map(s => s.length)
val totalLength = lineLengths.reduce((a, b) => a + b)
```

- Other transformations and actions:
 - union(), intersection(), distinct()
 - count(), first(), take(), foreach()
 - ...
- Can specify if an RDD should be "persisted" to disk
 - Allows for faster recovery during cluster faults

Spark Overview [Zaharia et al., 2010]



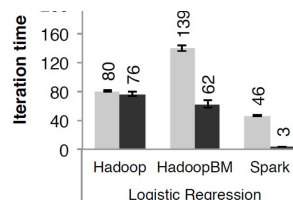
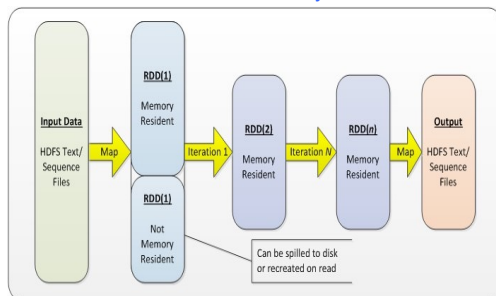
- Benefits of Spark:
 - Fault tolerant - RDDs immutable, just re-compute from lineage
 - Cacheable - keep some RDDs in RAM
 - Faster than Hadoop MR at iterative algorithms
 - Supports MapReduce as special case



Spark: Faster MapR on Data-Parallel



- Spark's solution: **Resilient Distributed Datasets (RDDs)**
 - Input data → load as RDD → apply transforms → output result
 - RDD transforms strict superset of MapR
 - RDDs cached in memory, avoid disk I/O

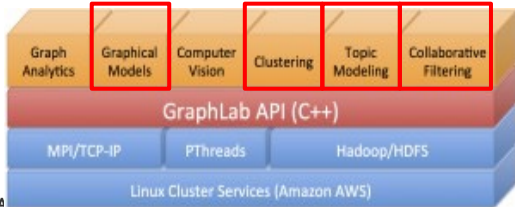


- Spark ML library supports data-parallel ML algos, like Hadoop
 - Spark and Hadoop: comparable first iter timings...
 - But Spark's later iters are much faster



GraphLab Overview [Low et al., 2012]

- Known as “GraphLab PowerGraph v2.2”
 - Different from commercial software “GraphLab Create” by Dato.com, who formerly developed PowerGraph v2.2
- System for Graph Programming
 - Think of ML algos as graph algos
- Comes with ready-to-run “toolkits”
 - ML-centric toolkits: clustering, collaborative filtering, topic modeling, graphical models





ĐẠI HỌC BÁCH KHOA
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

31

31



GraphLab Overview [Low et al., 2012]

- ML-related toolkits
 - Clustering
 - K-means
 - Spectral
 - Collaborative Filtering
 - Matrix Factorization (including Non-negative, L1/L2-regularized)
 - Graphical Models
 - Factor graphs
 - Belief propagation algorithm
 - Topic Modeling
 - LDA
- Other toolkits available for computer vision, graph analytics, linear systems



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

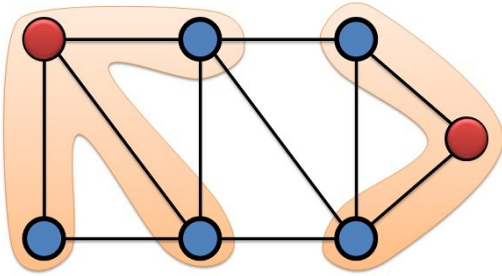
32

32



GraphLab Overview [Low et al., 2012]

- Key feature: Gather-Apply-Scatter Programming Model
 - Write ML algos as **vertex programs**
 - Run vertex programs in parallel on each graph node
 - Graph nodes, edges can have data, parameters





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Source: Gonzalez (2012)

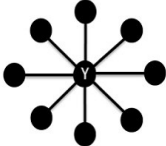
33

33

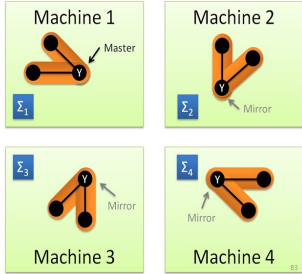


GraphLab Overview [Low et al., 2012]

- Programming Model: GAS Vertex Programs
 - 1) Gather():** Accumulate data, params from my neighbors + edges
 - 2) Apply():** Transform output of Gather(), write to myself
 - 3) Scatter():** Transform output of Gather(), Apply(), write to my edges


→

Gather





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Source: Gonzalez (2012)

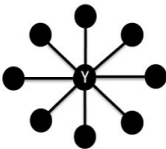
34

34



GraphLab Overview [Low et al., 2012]

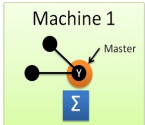
- Programming Model: GAS Vertex Programs
 - 1) Gather(): Accumulate data, params from my neighbors + edges
 - 2) **Apply()**: Transform output of Gather(), write to myself
 - 3) Scatter(): Transform output of Gather(), Apply(), write to my edges


→

Gather

Apply

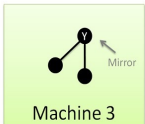
Machine 1



Machine 2



Machine 3



Machine 4



Source: Gonzalez (2012)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

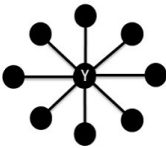
35

35



GraphLab Overview [Low et al., 2012]

- Programming Model: GAS Vertex Programs
 - 1) Gather(): Accumulate data, params from my neighbors + edges
 - 2) Apply(): Transform output of Gather(), write to myself
 - 3) **Scatter()**: Transform output of Gather(), Apply(), write to my edges

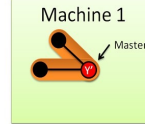

→

Gather

Apply

Scatter

Machine 1



Machine 2



Machine 3



Machine 4



Source: Gonzalez (2012)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

36

36



GraphLab Overview [Low et al., 2012]

- Example GAS program: Pagerank
 - Programmer implements `gather()`, `apply()`, `scatter()` functions

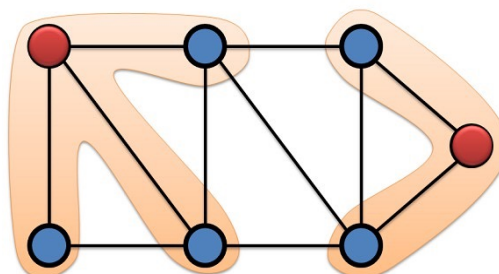
```
// gather_nbrs: IN_NBRs
gather(D_u, D_{u,v}, D_v):
    return D_v.rank / #outNbrs(v)
sum(a, b): return a + b
apply(D_u, acc):
    rnew = 0.15 + 0.85 * acc
    D_u.delta = (rnew - D_u.rank) /
                #outNbrs(u)
    D_u.rank = rnew
// scatter_nbrs: OUT_NBRs
scatter(D_u, D_{u,v}, D_v):
    if(|D_u.delta| > ε) Activate(v)
    return delta
```

Source: Gonzalez et al. (OSDI 2012)



GraphLab Overview [Low et al., 2012]

- Benefits of Graphlab
 - Supports asynchronous execution - fast, avoids straggler problems
 - Edge-cut partitioning - scales to large, power-law graphs
 - Graph-correctness - for ML, more fine-grained than MapR-correctness

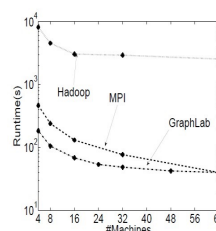
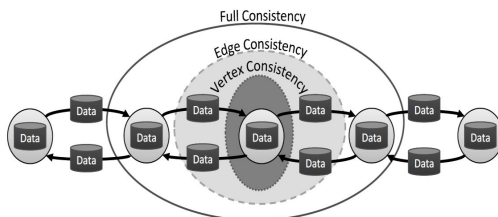


Source: Gonzalez (2012)

GraphLab: Model-Parallel via Graphs



- GraphLab Graph consistency models
 - Guide search for "ideal" model-parallel execution order
 - ML algo correct if input graph has all dependencies

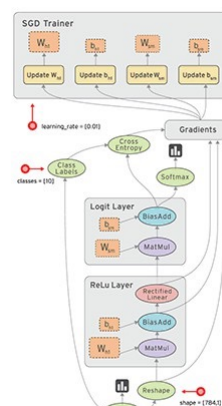


- GraphLab supports asynchronous (no-waiting) execution
 - Correctness enforced by graph consistency model
 - Result: GraphLab graph-parallel ML much faster than Hadoop

Google TensorFlow: Dataflow-style system

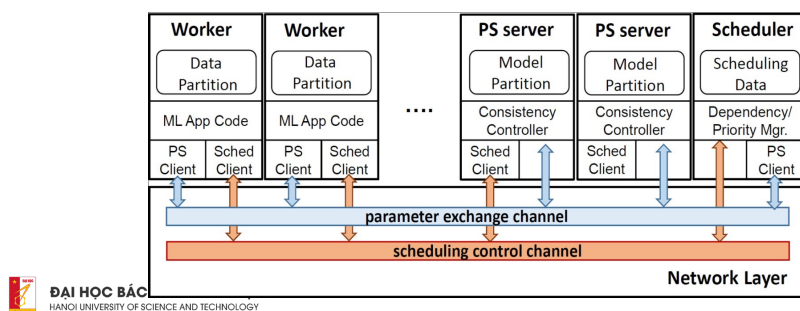


- First release Nov 2015
- Auto-differentiation + Dataflow system
 - Conceptually similar to Spark RDDs
 - Geared towards tensor/matrix computation
 - Asynchronous execution
- Distributed support is Work-in-Progress
 - Results are mixed, performance is OK for smaller models
 - Large models do not benefit from going distributed, e.g. VGG



Parallel ML System Overview (Formerly Petuum) [Xing et al., 2015]

- Key modules
 - **Key-value store** (Parameter Server) for data-parallel ML algos
 - **Scheduler** for model-parallel ML algos
- Program ML algos in iterative-convergent style
 - ML algo = (1) write update equations + (2) iterate eqns via schedule

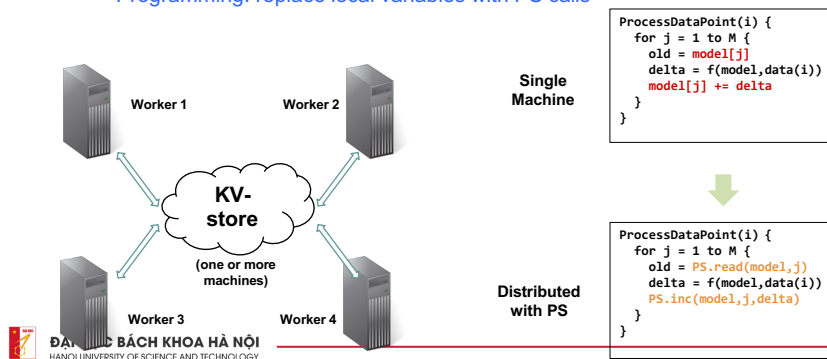


41

41

PMLS Overview [Xing et al., 2015]

- Key-Value store (Parameter Server)
 - Enables **data-parallelism**
 - A type of **Distributed Shared Memory (DSM)**
 - Model parameters **globally shared across workers**
 - Programming: replace local variables with PS calls

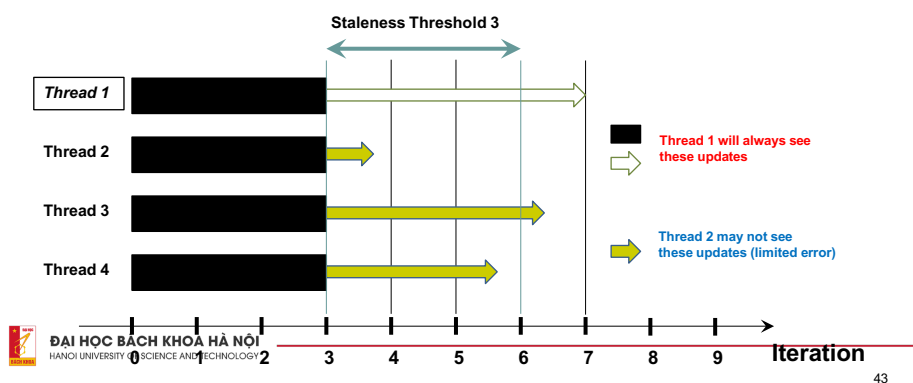


42

42

PMLS Overview [Xing et al., 2015]

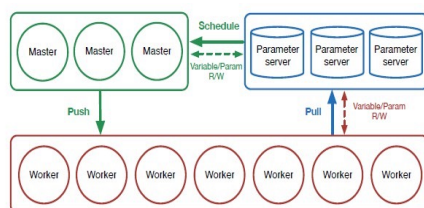
- Key-Value store features:
 - ML-tailored consistency model: Stale Synchronous Parallel (SSP)
 - Asynchronous-like speed
 - Bulk Synchronous Parallel-like correctness guarantees for ML



43

PMLS Overview [Xing et al., 2015]

- Scheduler
 - Enables correct model-parallelism
 - Can analyze ML model structure for best execution order
 - Programming: `schedule()`, `push()`, `pull()` abstraction



```

schedule() {
  // Select U vars x[j] to be sent
  // to the workers for updating
  ...
  return (x[j_1], ..., x[j_U])
}

push(worker = p, vars = (x[j_1], ..., x[j_U])) {
  // Compute partial update z for U vars x[j]
  // at worker p
  ...
  return z
}

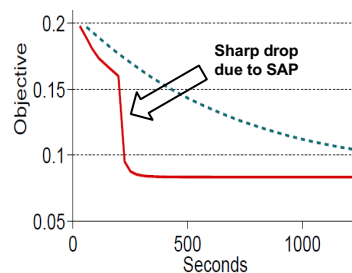
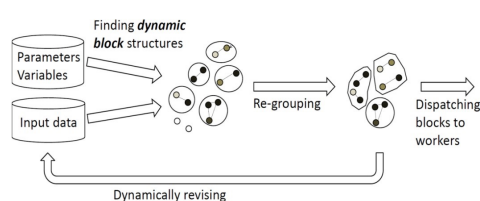
pull(workers = [p], vars = (x[j_1], ..., x[j_U]),
      updates = [z]) {
  // Use partial updates z from workers p to
  // update U vars x[j]. sync() is automatic.
  ...
}

```

44

PMLS Overview [Xing et al., 2015]

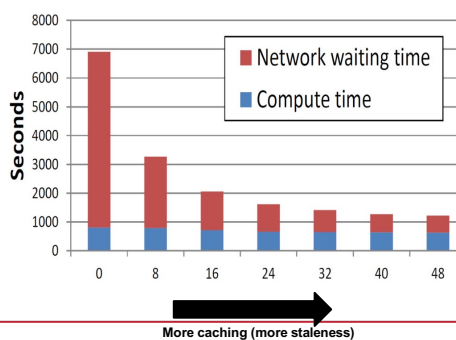
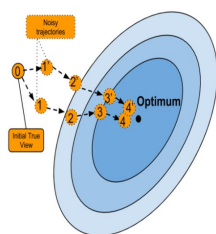
- Scheduler benefits:
 - ML scheduling engine: Structure-Aware Parallelization (SAP)
 - Scheduled ML algos require less computation to finish



45

PMLS: ML props = 1st-class citizen

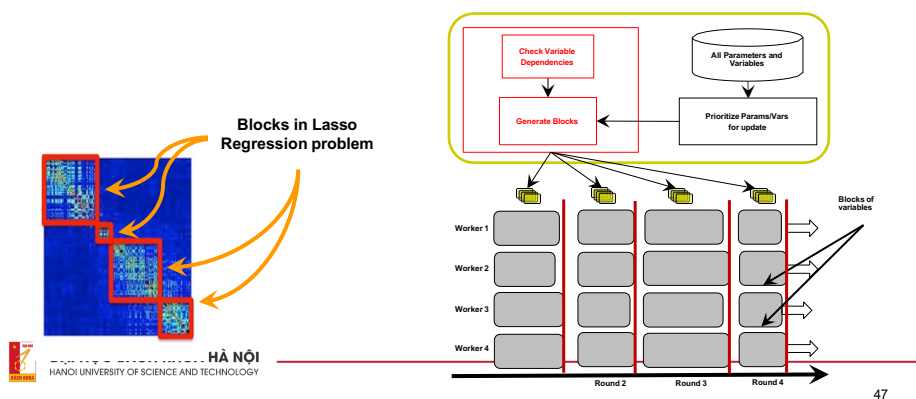
- Error tolerance via Stale Sync Parallel KV-store
 - System Insight 1: ML algos bottleneck on network comms
 - System Insight 2: More caching => less comms => faster execution



46

PMLS: ML props = 1st-class citizen

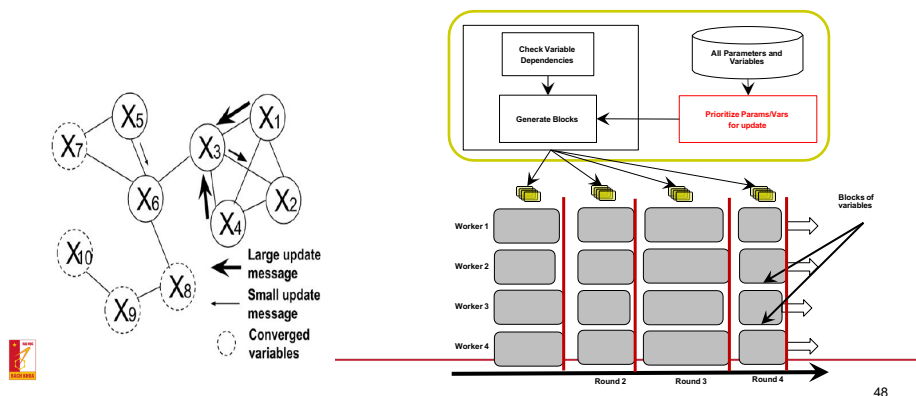
- Harness Block dependency structure via Scheduler
 - System Insight 1: Pipeline scheduler to hide latency
 - System Insight 2: Load-balance blocks to prevent stragglers



47

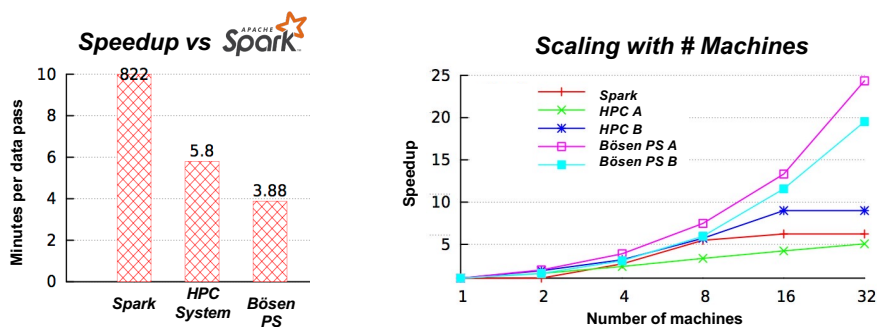
PMLS: ML props = 1st-class citizen

- Exploit Uneven Convergence via Prioritizer
 - System Insight 1: Prioritize small # of vars => fewer deps to check
 - System Insight 2: Lowers computational cost of Scheduling



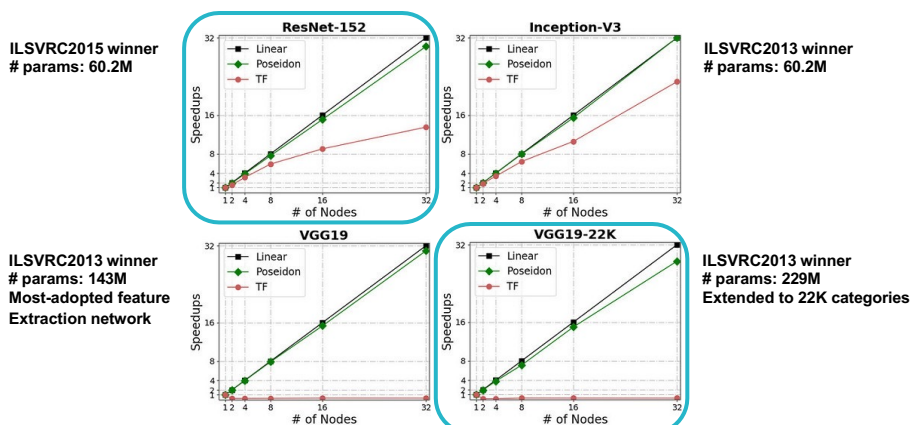
48

PMLS Research in 2016: Parameter Servers



Task: SGD Matrix Factorization, 32 machines (16 cores, 32GB RAM), 250M parameters, 1.3GB data

PMLS Research in 2016: Deep Learning Systems



ML Programming Interface: Needs and Considerations

- An ideal ML programming interface should make it easy to write **correct** data-parallel, model-parallel ML programs
- What can be abstracted away?
 - Abstract away inter-worker communication/synchronization:
 - Automatic consistency models; bandwidth management through distributed shared memory
 - Abstract scheduling away from update equations:
 - Easy to change scheduling strategy, or use dynamic schedules
 - Abstract away worker management:
 - Let ML system decide optimal number and configuration of workers
 - Ideally, reduce programmer burden to just 3 things:
 - Declare model, write updates, write schedule

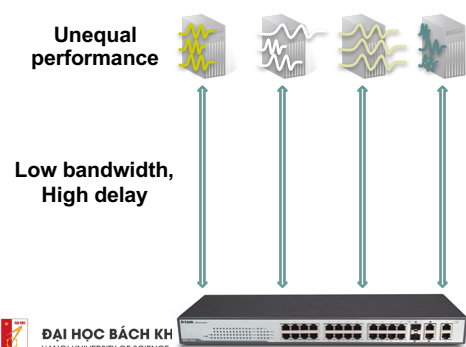
51

Systems, Architectures for Distributed ML

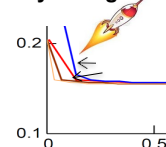
52

There Is No Ideal Distributed System!

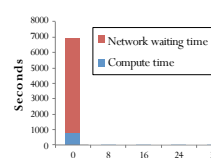
- Not quite that easy...
- **Two distributed challenges:**
 - Networks are slow
 - "Identical" machines rarely perform equally



Async execution:
May diverge



BSP execution:
Long sync time



53

53

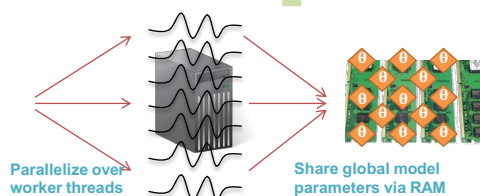
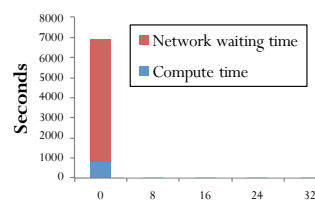
Why need new Big ML systems?

MLer's view

- Focus on
 - Correctness
 - fewer iteration to converge,
- but assuming an ideal system, e.g.,
 - zero-cost sync,
 - uniform local progress

```
for (t = 1 to T) {
  doThings()
  parallelUpdate(x, θ)
  doOtherThings()
}
```

Compute vs Network
LDA 32 machines (256 cores)

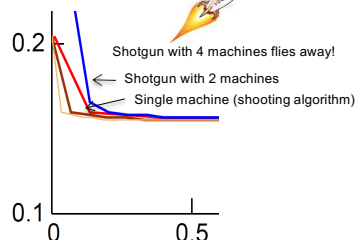


54

54

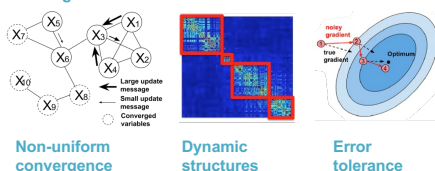
Why need new Big ML systems?

Systems View:

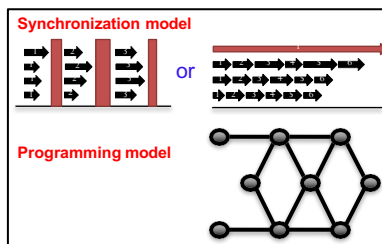


- Focus on
 - high iteration throughput (more iter per sec)
 - strong fault-tolerant atomic operations,
- but assume ML algo is a black box
 - ML algos "still work" under different execution models
 - "easy to rewrite" in chosen abstraction

Agonistic of ML properties and objectives in system design



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



55

55

Why need new Big ML systems?

MLer's view

- Focus on
 - Correctness
 - fewer iteration to converge,
- but assuming an ideal system, e.g.,
 - zero-cost sync,
 - uniform local progress

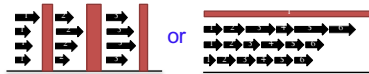
```
for (t = 1 to T) {
  doThings()
  parallelUpdate(x, θ)
  doOtherThings()
}
```

Oversimplify systems issues

- need machines to perform consistently
- need lots of synchronization
- or even try not to communicate at all

Systems View:

- Focus on
 - high iteration throughput (more iter per sec)
 - strong fault-tolerant atomic operations,
- but assume ML algo is a black box
 - ML algos "still work" under different execution models
 - "easy to rewrite" in chosen abstraction



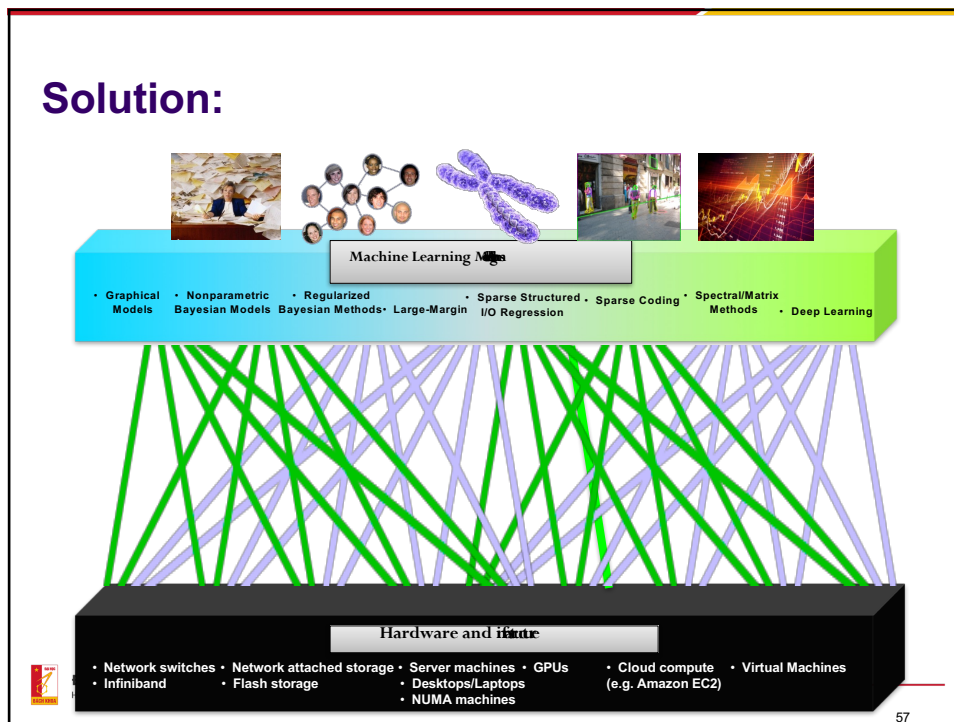
Oversimplify ML issues and/or ignore ML opportunities

- ML algos "just work" without proof
- Conversion of ML algos across different program models (graph programs, RDD) is easy

56

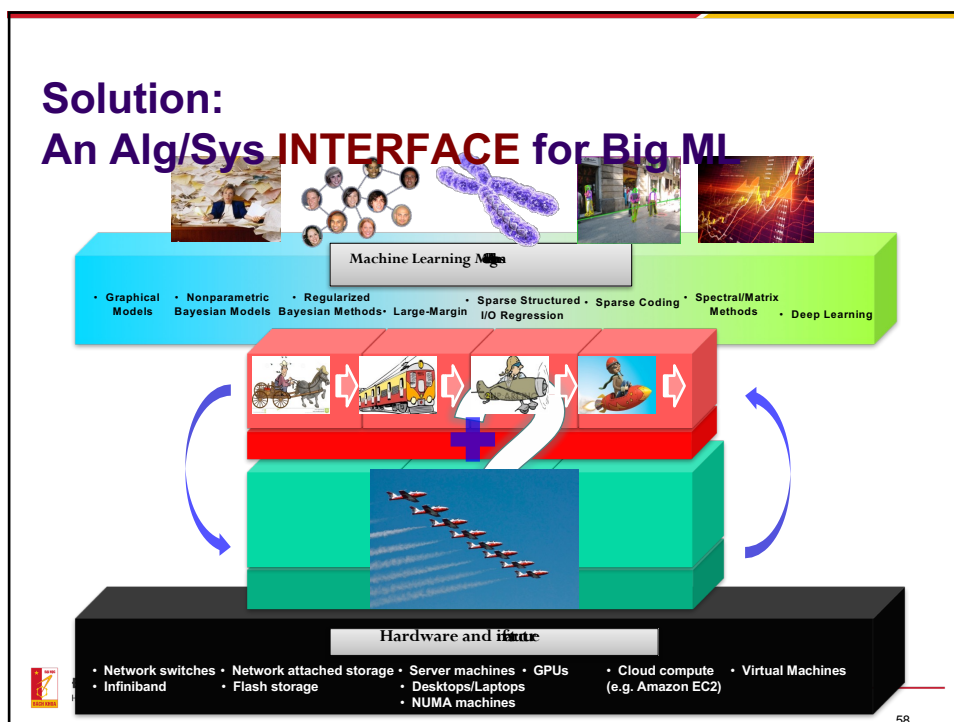
56

Solution:



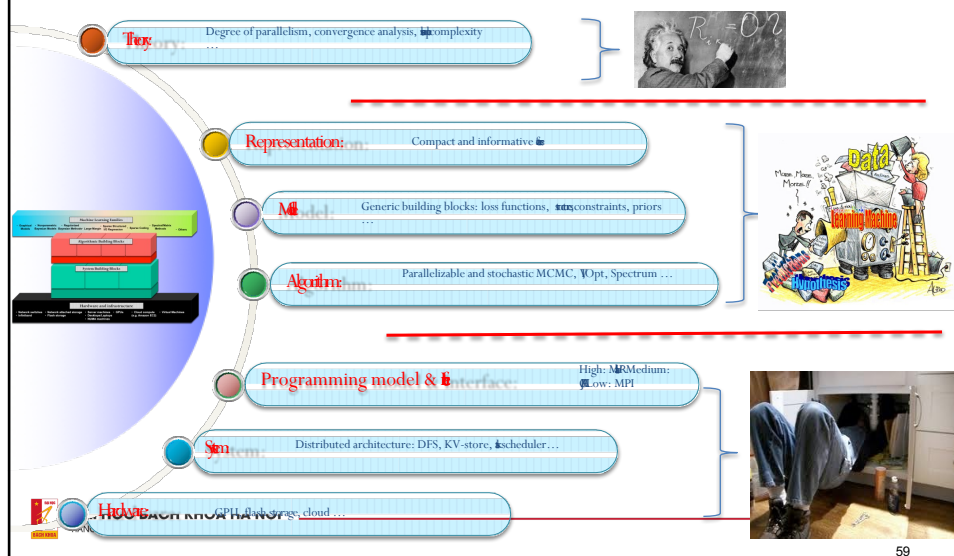
57

Solution: An Alg/Sys INTERFACE for Big ML



58

The Big-ML “Stack” - More than just software

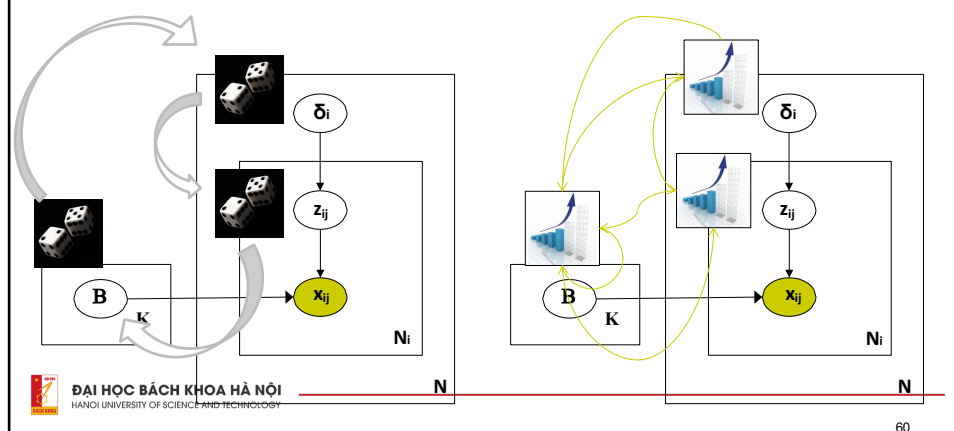


59

ML algorithms are Iterative-Convergent

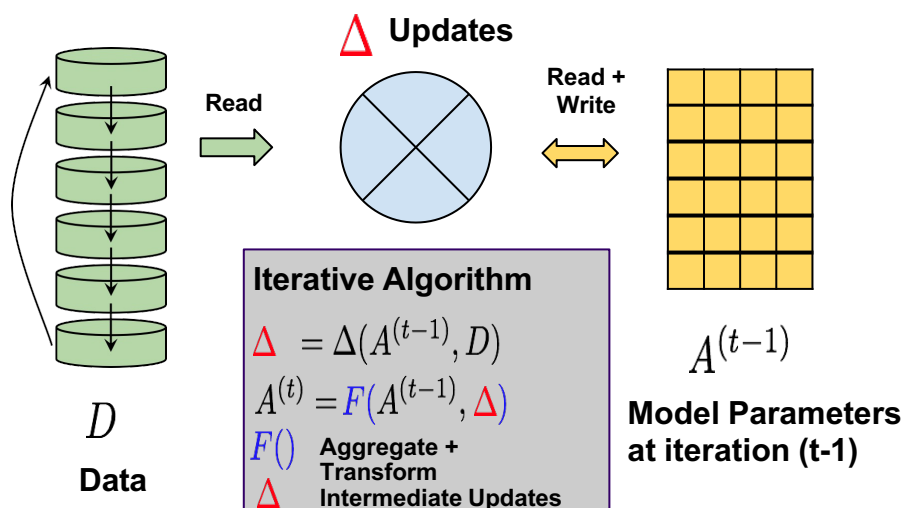
Markov Chain Monte Carlo

Optimization



60

A General Picture of ML Iterative-Convergent Algorithms



Issues with Hadoop and I-C ML Algorithms?

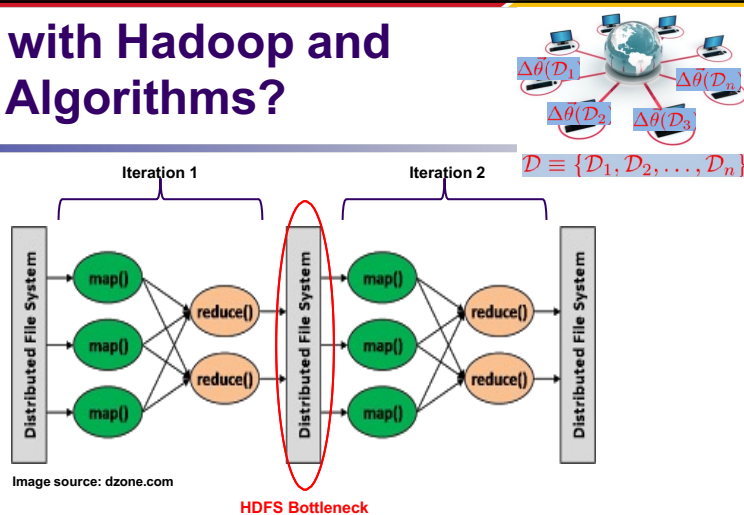
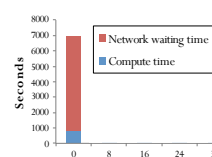
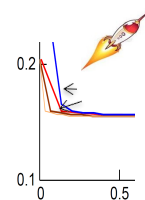
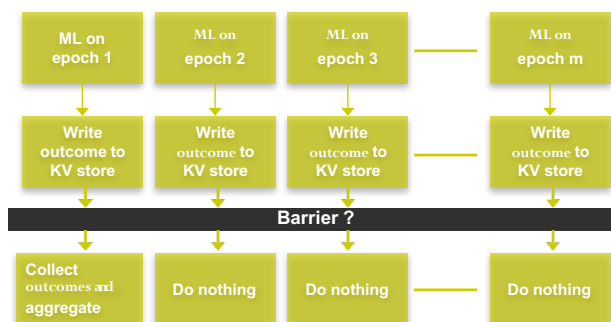


Image source: dzone.com

Naïve MapReduce not best for ML

- Hadoop can execute iterative-convergent, data-parallel ML...
 - `map()` to distribute data samples i , compute update $\Delta(D_i)$
 - `reduce()` to combine updates $\Delta(D_i)$
 - Iterative ML algo = repeat `map()`+`reduce()` again and again
- But reduce() writes to HDFS before starting next iteration's `map()` - very slow iterations!

Good Parallelization Strategy is important



```

for (t = 1 to T) {
  doThings()
  parallelUpdate(x, θ)
  doOtherThings()
}
  
```

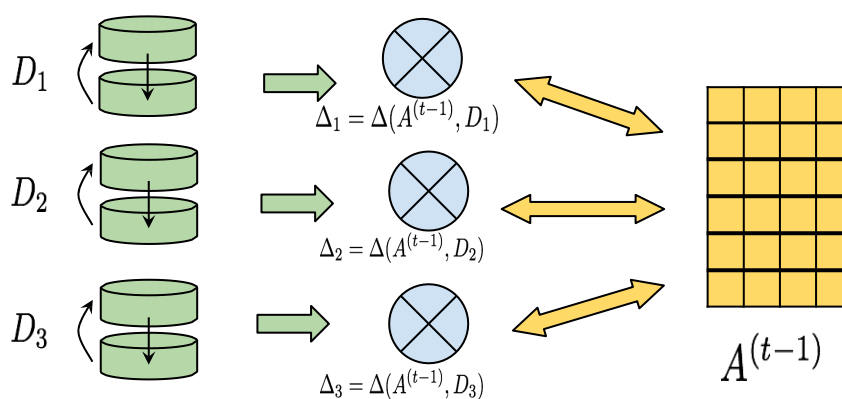


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

63

63

Data Parallelism



Additive Updates

$$\Delta = \sum_{p=1}^3 \Delta_p$$

$$A^{(t)} = F(A^{(t-1)}, \Delta)$$

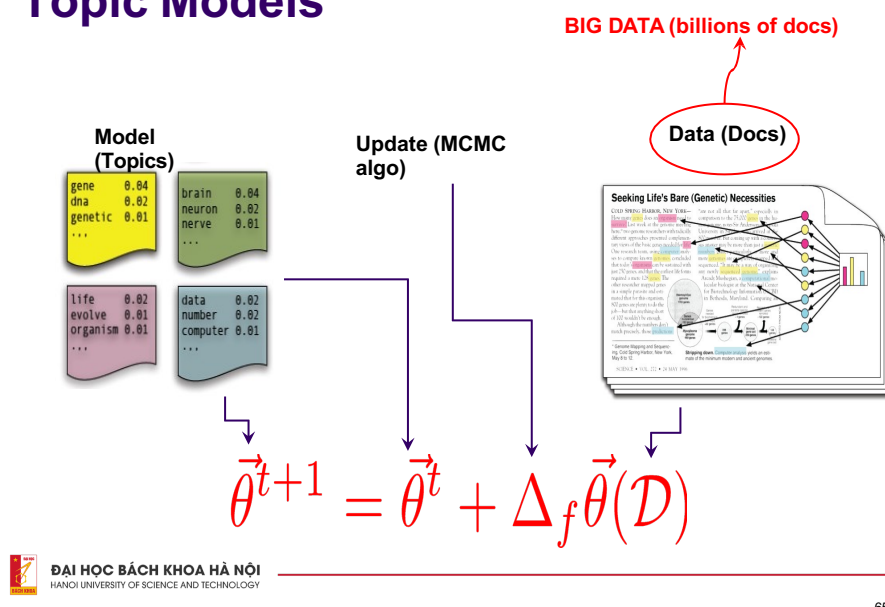


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

64

64

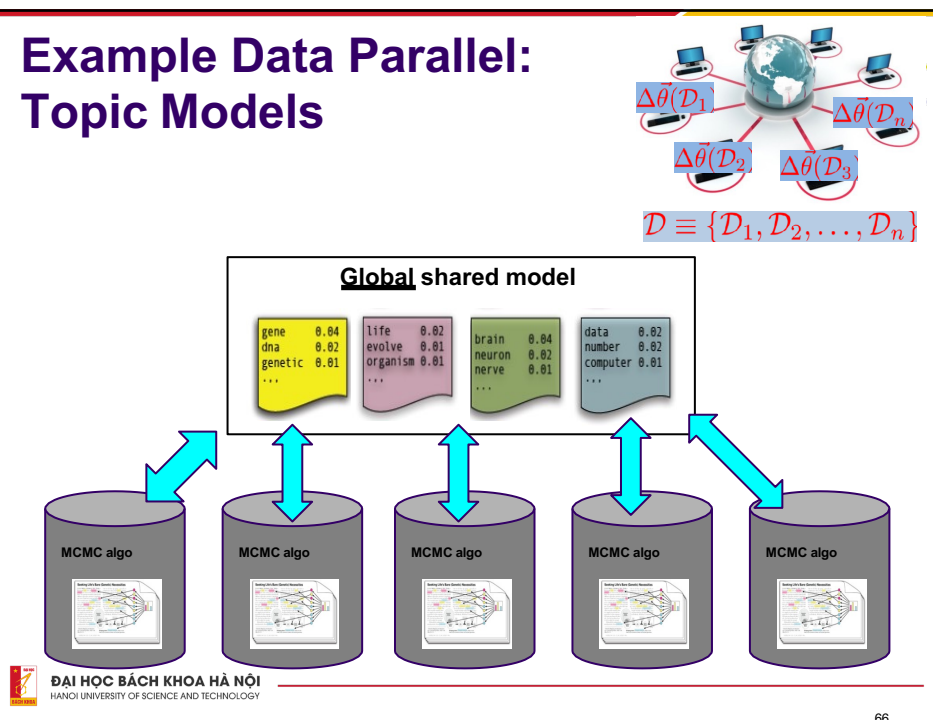
Example Data Parallel: Topic Models



65

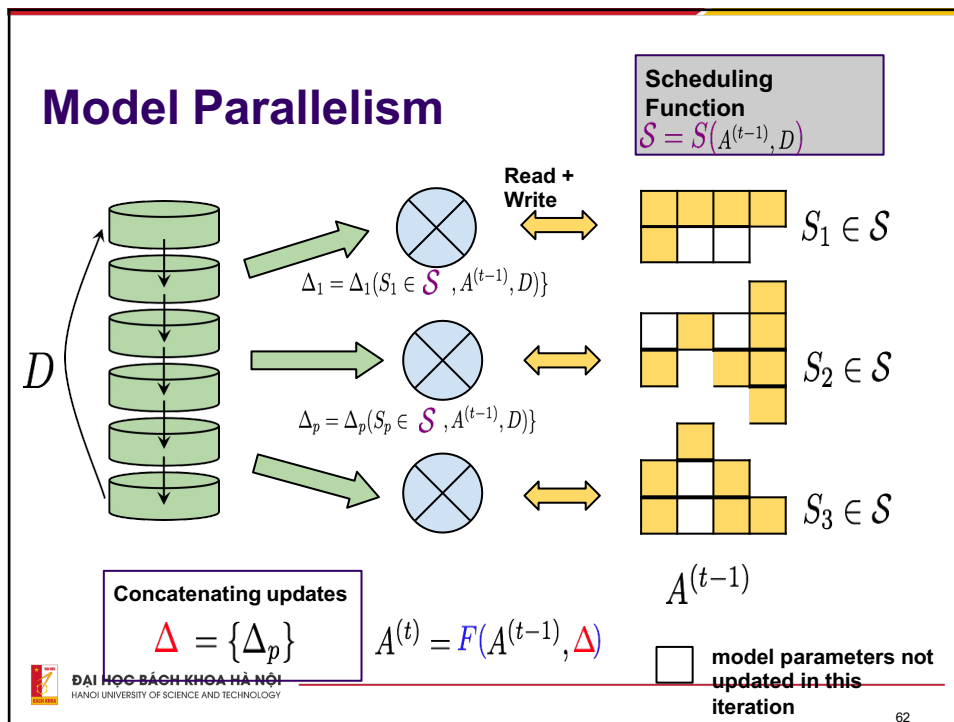
65

Example Data Parallel: Topic Models

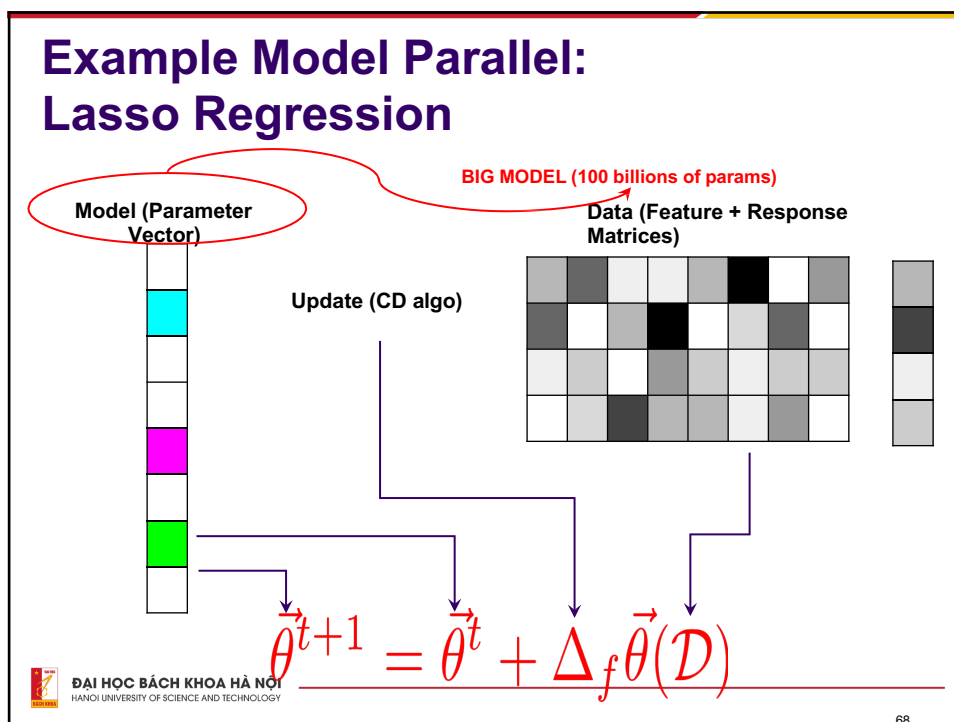


66

66

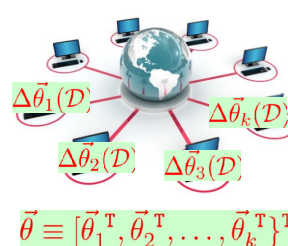


67

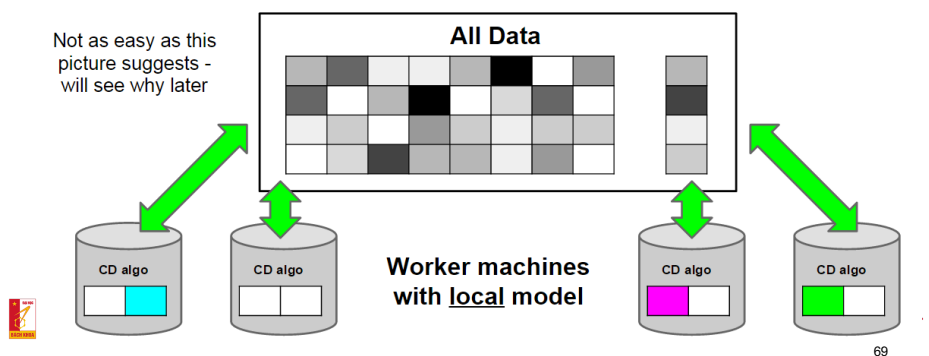


68

Example Model Parallel: Lasso Regression

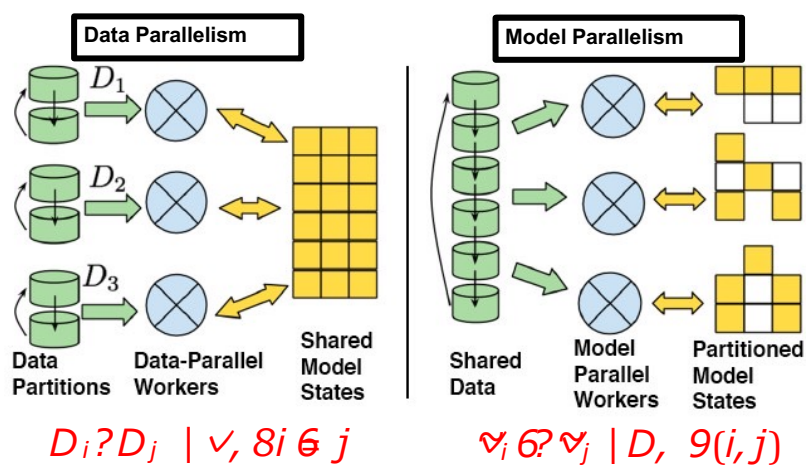


Not as easy as this picture suggests - will see why later

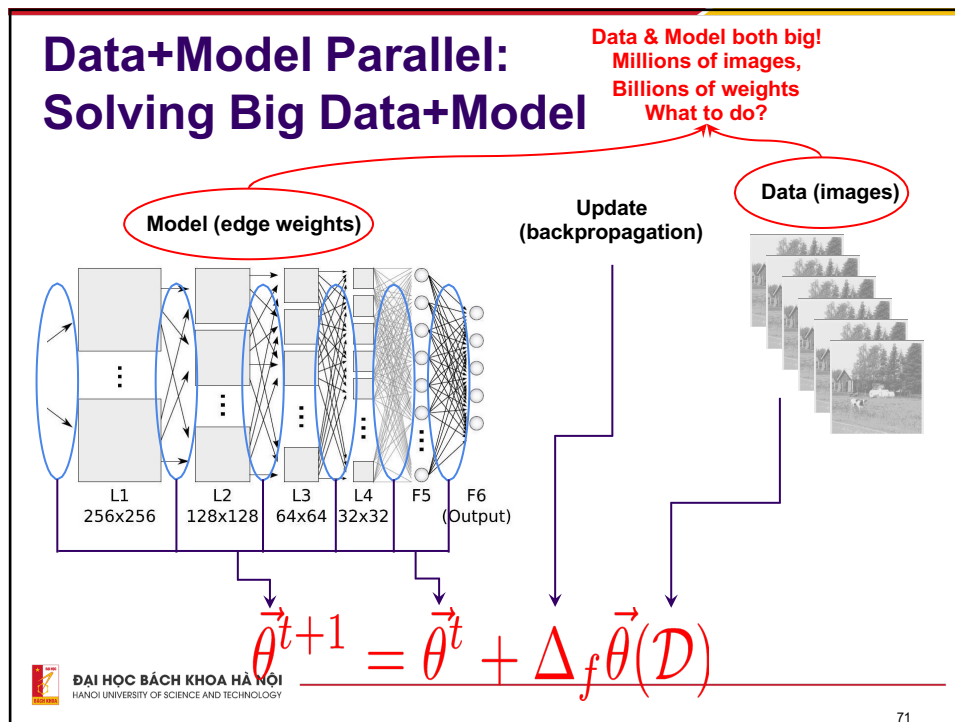


69

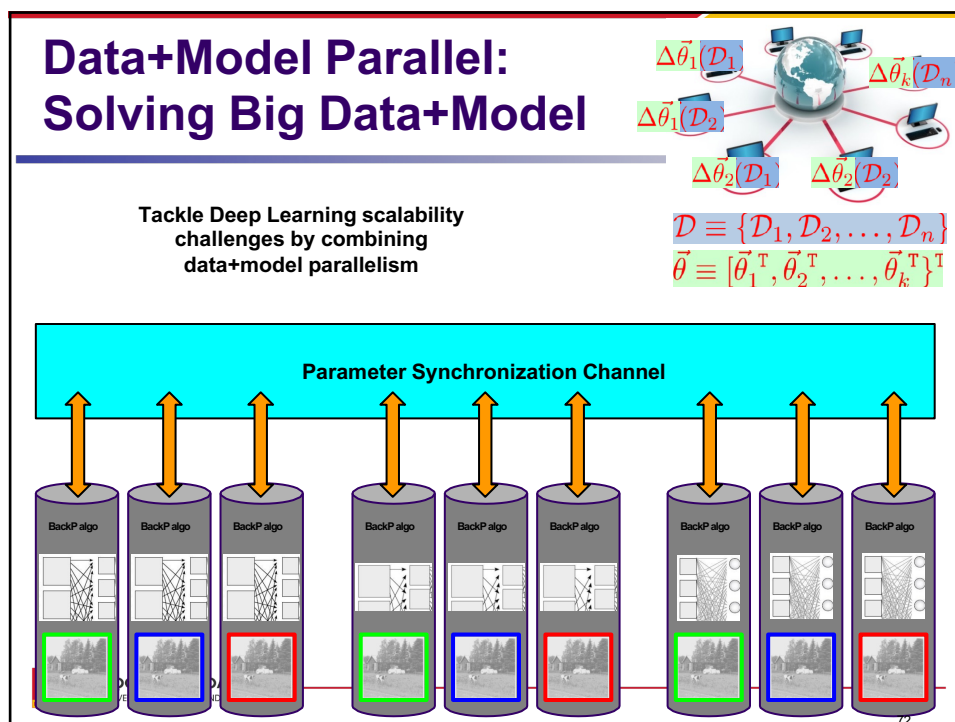
A Dichotomy of Data and Model in ML Programs



70



71



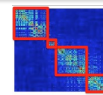
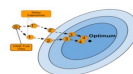
72

How difficult is data/model-parallelism?

- Certain **mathematical** conditions must be met
- Data-parallelism generally OK when data IID (independent, identically distributed)
 - Very close to serial execution, in most cases
- Naive Model-parallelism doesn't work
 - NOT equivalent to serial execution of ML algo
 - Need carefully designed **schedule**

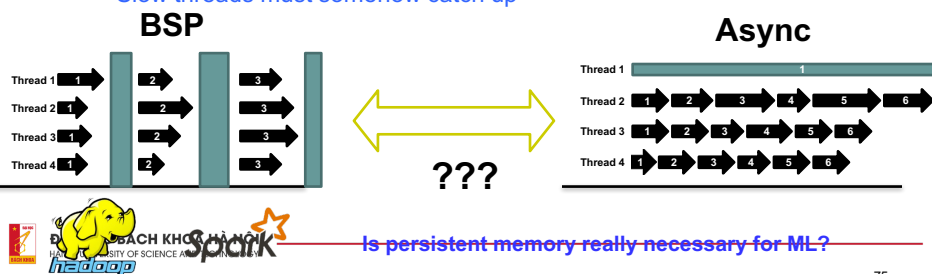
Intrinsic Properties of ML Programs

- ML is **optimization-centric**, and admits an **iterative convergent** algorithmic solution rather than a one-step closed form solution
 - **Error tolerance**: often robust against limited errors in intermediate calculations
- **Dynamic structural dependency**: changing correlations between model parameters critical to efficient parallelization
- **Non-uniform convergence**: parameters can converge in very different number of steps
- Whereas traditional programs are **transaction-centric**, thus only guaranteed by **atomic correctness** at every step
- Most existing platforms (e.g., Spark, GraphLab) have not yet systematically explored and exploit above properties



Challenges in Data Parallelism

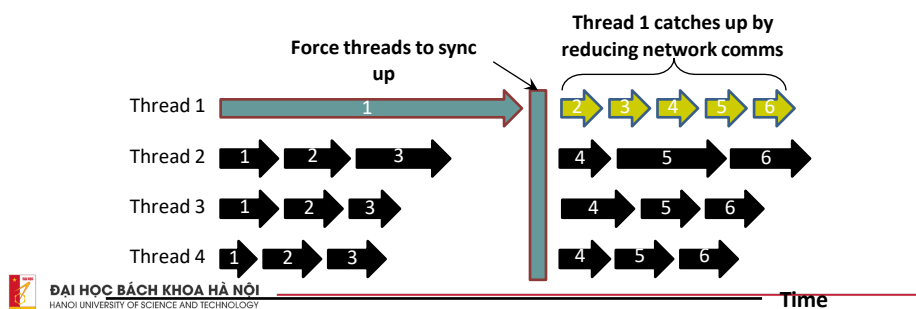
- Existing ways are either safe/slow (BSP), or fast/risky (Async)
- Challenge 1: Need “Partial” synchronicity**
 - Spread network comms evenly (don't sync unless needed)
 - Threads usually shouldn't wait – but mustn't drift too far apart!
- Challenge 2: Need straggler tolerance**
 - Slow threads must somehow catch up



75

Is there a middle ground for data-parallel consistency?

- Challenge 1: “Partial” synchronicity**
 - Spread network comms evenly (don't sync unless needed)
 - Threads usually shouldn't wait – but mustn't drift too far apart!
- Challenge 2: Straggler tolerance**
 - Slow threads must somehow catch up



76

High-Performance Consistency Models for Fast Data-Parallelism [Ho et al., 2013]



Stale Synchronous Parallel (SSP), a “bounded-asynchronous” model

- Allow threads to run at their own pace, without synchronization
- Fastest/slowest threads not allowed to drift $>S$ iterations apart
- Threads cache local (stale) versions of the parameters, to reduce network syncing

Consequence:

- Asynchronous-like speed, BSP-like ML correctness guarantees
- Guaranteed age bound (staleness) on reads



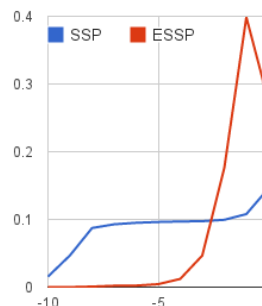
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

77

77

Improving Bounded-Async via Eager Updates [Dai et al., 2015]

- Eager SSP (ESSP) protocol
 - Use spare bandwidth to push fresh parameters sooner
- Figure: difference in stale reads between SSP and ESSP
 - ESSP has fewer stale reads; lower staleness variance
 - Faster, more stable convergence (theorems later)



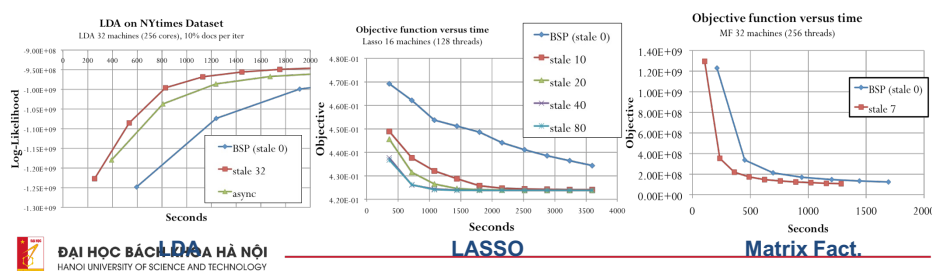
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

78

78

Enjoys Async Speed, yet BSP Guarantee, across algorithms

- Scale up Data Parallelism without being limited by long BSP synchronization time
- Effective across different algorithms, e.g. LDA, Lasso, Matrix Factorization:



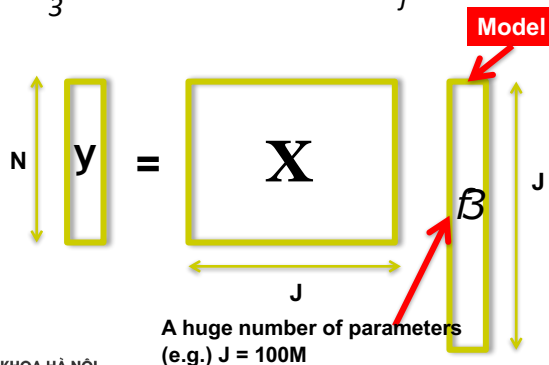
79

79

Challenges in Model Parallelism

- Recall Lasso regression:

$$\min_{\beta} \|y - X\beta\|_2^2 + \lambda \sum_j |\beta_j|$$



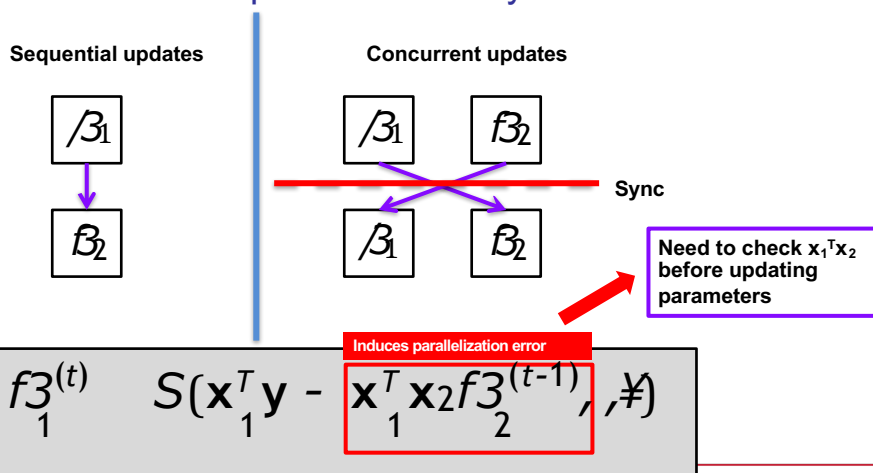
DAI HỌC BACH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

80

80

Challenge 1: Model Dependencies

- Concurrent updates of β may induce errors

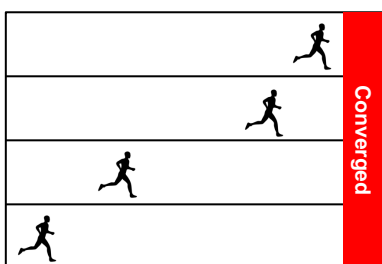


81

81

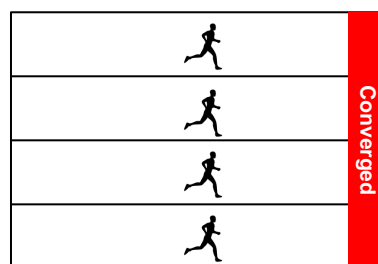
Challenge 2: Uneven Convergence Rate on Parameters

Parameters converge at different rates



Remaining time to convergence

Parameters converge at similar rates



Remaining time to convergence

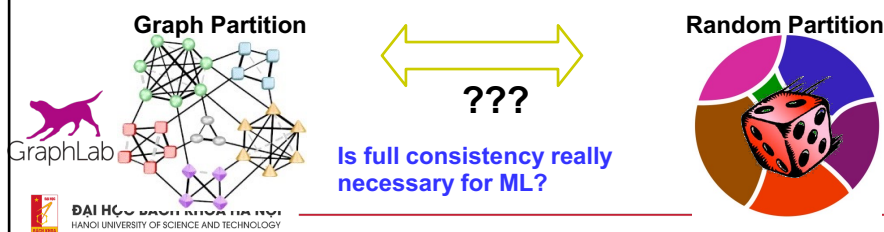
- Convergence time determined by slowest parameters
- How to make slowest parameters converge more quickly?

77

82

Is there a middle ground for model-parallel consistency?

- Existing ways are either safe but slow, or fast but risky
- **Challenge 1: need approximate but fast model partition**
 - Full representation of data/model, and explicitly compute all dependencies via graph cut is not feasible
- **Challenge 2: need dynamic load balancing**
 - Capture and explore transient model dependencies
 - Explore uneven parameter convergence



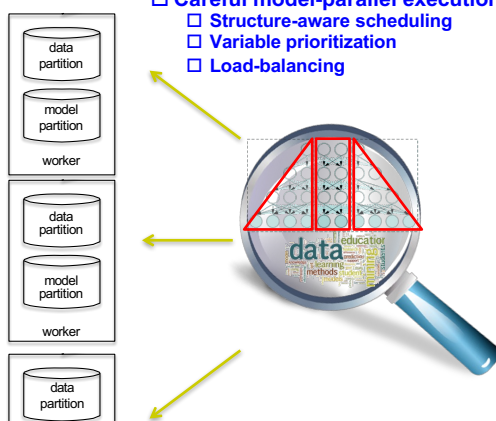
83

83

Structure-Aware Parallelization (SAP) [Lee et al., 2014; Kumar et al., 2014]

- Careful model-parallel execution:
- Structure-aware scheduling
 - Variable prioritization
 - Load-balancing

- Simple programming:
- Schedule()
 - Push()
 - Pull()



```

schedule() {
  // Select U vars x[j] to be sent
  // to the workers for updating
  ...
  return (x[j_1], ..., x[j_U])
}

push(worker = p, vars = (x[j_1], ..., x[j_U])) {
  // Compute partial update z for U vars x[j]
  // at worker p
  ...
  return z
}

pull(workers = [p], vars = (x[j_1], ..., x[j_U]),
      updates = [z]) {
  // Use partial updates z from workers p to
  // update U vars x[j]. sync() is automatic.
  ...
}

```

DAI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

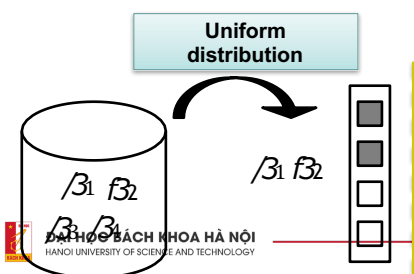
84

84

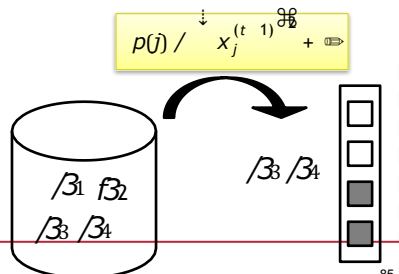
Schedule 1: Priority-based [Lee et al., 2014]

- Choose params to update based on convergence progress
 - Example: sample params with probability proportional to their recent change
 - Approximately maximizes the convergence progress per round

Shotgun [Bradley et al. 2011]



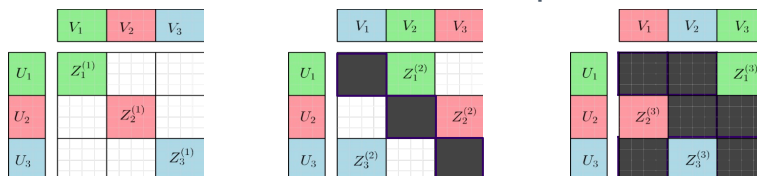
Priority-based scheduling



85

Schedule 2: Block-based (with load balancing) [Kumar et al., 2014]

Partition data & model into $d \times d$ blocks
Run different-colored blocks in parallel

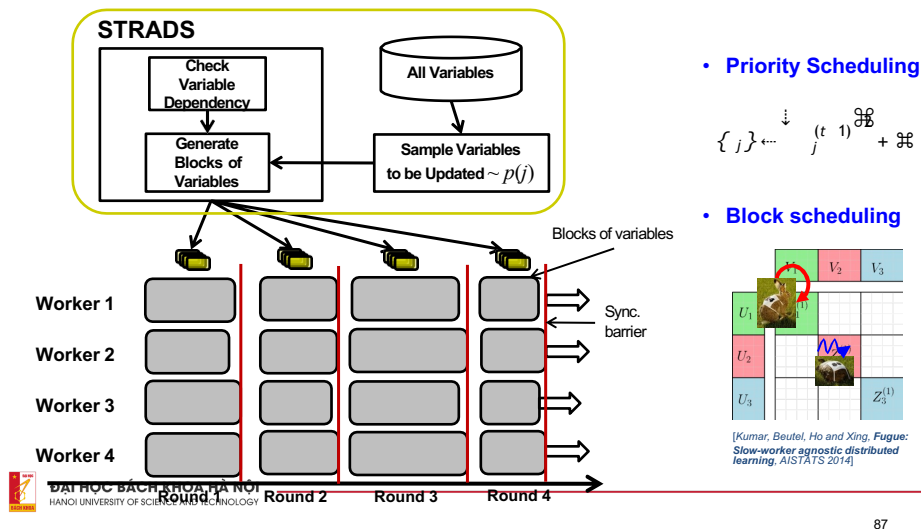


Blocks with less data/para or experience less straggling run more iterations
Automatic load-balancing + better convergence

86

86

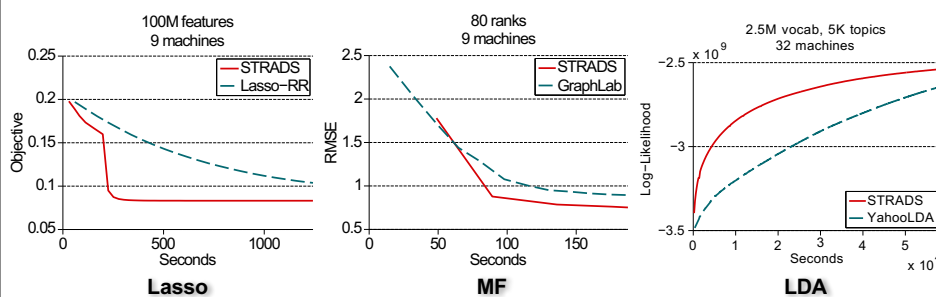
Structure-aware Dynamic Scheduler (STRADS) [Lee et al., 2014, Kumar et al., 2014]



87

Avoids dependent parallel updates, attains near-ideal convergence speed

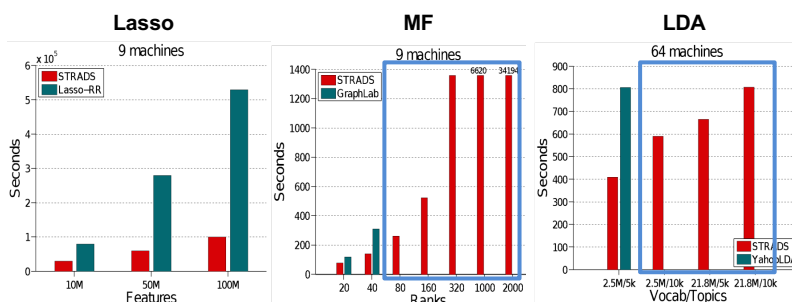
- STRADS+SAP achieves better speed and objective



88

Efficient for large models

- Model is partitioned => can run larger models on same hardware



Theory of Real Distributed ML Systems

Why study parallel ML theory?

- What sequential guarantees still hold in parallel setting?
 - Under what conditions?
- Growing body of literature for “ideal” parallel systems
 - Serializable— equivalent to single-machine execution in some sense
 - Focused on per-iteration analysis
 - Abstract away computational/comms cost
 - Predicting real-world running time requires these costs to be put back
- “Real-world” parallel systems a work in progress
 - Asynchronous or bounded-async approaches can empirically work better than synchronous approaches
 - Need additional theoretical analysis to understand why
 - Async => no serializability... why does it still work?
 - Parallelization requires data and/or model partitioning... many strategies exist
 - Want partitioning strategies that are provably **correct**
 - Need to determine when/where independence is violated, and what impact such violation has on algorithm correctness

Challenges in real-world distributed systems

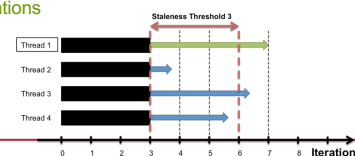
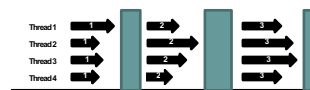
- Real-world systems need asynchronous execution and load balancing
 - Synchronous system: load imbalances => slow workers => waiting at barriers
 - Need load balancing to reduce load at slow workers
 - Need asynchronous execution so faster workers can proceed without waiting
- Solution 1: key-value stores
 - Automatically manages communication with bounded asynchronous guarantees
- Solution 2: scheduling systems
 - Automatically balances workload across workers; also performs prioritization and dependency checking

Communication strategies

- Data parallel
 - Partition data across workers
 - Or fetch small batches of data in an online/streaming fashion
 - Communicate model as needed to workers
 - e.g. key-value store with bounded asynchronous model – theoretical consequences?
- Model parallel
 - Partition model across workers
 - Model partitions can change dynamically during execution – theoretical consequences?
 - Send data to workers as needed (e.g. from shared database)
 - Or place full copy of data on each worker (since data is immutable)
- Data + Model parallel?
 - Partition both data and model across workers
 - Wide space of strategies; need to reduce model and data communication
 - Reduce model communication by exploiting independence between variables
 - Reduce data and model communication via broadcast strategies, e.g. Halton sequence

Bridging Models for Parallel Programming

- Bulk Synchronous Parallel [Valiant, 1990] is a bridging model
 - Bridging model specifies how/when parallel workers should compute, and how/when workers should communicate
 - Key concept: barriers
 - No communication before barrier, only computation
 - No computation inside barrier, only communication
 - Computation is “serializable” – many sequential theoretical guarantees can be applied with no modification
- Bounded Asynchronous Parallel (BAP) bridging model
 - Key concept: bounded staleness [Ho et al., 2013; Dai et al., 2015]
 - Workers re-use old version of parameters, up to s iterations old – no need to barrier
 - Workers wait if parameter version older than s iterations

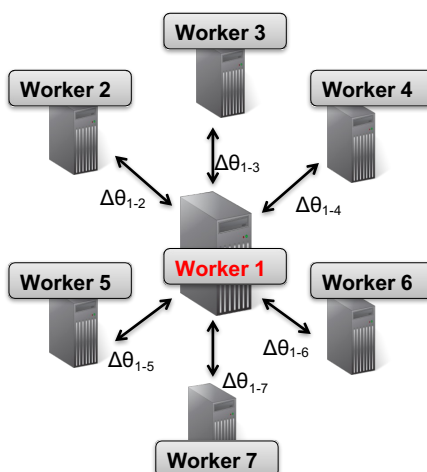


Types of Convergence Guarantees

- Regret/Expectation bounds on parameters
 - Better bounds => better convergence progress per iteration
- Probabilistic bounds on parameters
 - Similar meaning to regret/expectation bounds, usually stronger in guarantee
- Variance bounds on parameters
 - Lower variance => higher stability near optimum => easier to determine convergence
- For data parallel?
- For Model parallel?
- For Data + model parallel?

BAP Data Parallel: Can we do value-bounding?

- **Idea:** limit model parameter difference $\Delta\theta_{i,j} = \|\theta_i - \theta_j\|$ between machines i,j to $< \alpha$ threshold
- Does not work in practice!
 - To guarantee that $\Delta\theta_{i,j}$ has not exceeded the threshold, machines must wait to communicate with each other
 - No improvement over synchronous execution!
- Rather than controlling parameter difference via magnitude, what about via iteration count?
 - This is the (E)SSP communication model...



BAP Data Parallel: (E)SSP model

[Ho et al., 2013; Dai et al., 2015]



Stale Synchronous Parallel (SSP)

- Allow threads to run at their own pace, without synchronization
- Fastest/slowest threads not allowed to drift $>S$ iterations apart
- Threads cache local (stale) versions of the parameters, to reduce network syncing

Consequence:

- Asynchronous-like speed, BSP-like ML correctness guarantees
- **Guaranteed age bound (staleness) on reads**
- Contrast: no age guarantee Eventual Consistency seen in Cassandra, Memcached



DAI HO
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

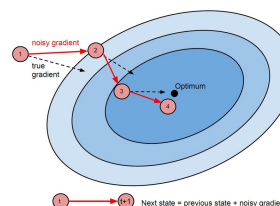
97

97

BAP Data Parallel: (E)SSP Regret Bound

[Ho et al., 2013]

- **Goal:** minimize convex $f(x) = \frac{1}{T} \sum_{t=1}^T f_t(x)$
(Example: Stochastic Gradient)
 - L -Lipschitz, problem diameter bounded by F^2
 - Staleness s , using P threads across all machines
 - Use step size $\eta_t = \frac{\sigma}{\sqrt{t}}$ with $\sigma = \frac{F}{L\sqrt{2(s+1)P}}$
- **(E)SSP converges according to**
 - Where T is the number of iterations



Difference between
SSP estimate and true optimum

$$R[X] := \left[\frac{1}{T} \sum_{t=1}^T f_t(\tilde{x}_t) \right] - f(x^*) \leq 4FL \sqrt{\frac{2(s+1)P}{T}}$$

- Note the RHS interrelation between (L, F) and (s, P)
- An interaction between model and systems parameters
- Stronger guarantees on means and variances can also be proven



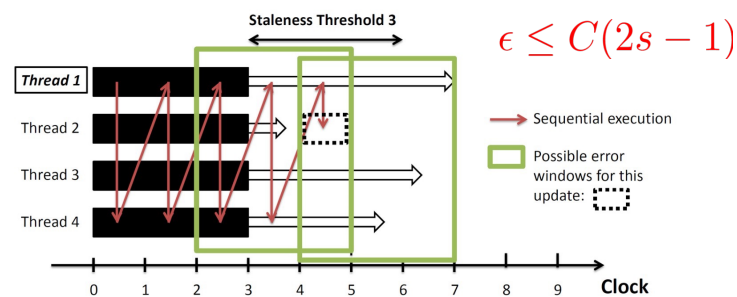
DAI HO
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

98

98

Intuition: Why does (E)SSP converge?

SSP approximates sequential execution



- Number of missing updates bounded
 - Partial, but bounded, loss of serializability
- Hence numeric error in parameter also bounded



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

End of this tutorial – formal theorem

99

99

SSP versus ESSP: What is the difference?

- ESSP is a systems improvement over SSP communication
 - Same maximum staleness guarantee as SSP
 - Whereas SSP waits until the last second to communicate...
 - ... ESSP communicates updates as early as possible
- What impact does ESSP have on convergence speed and stability?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

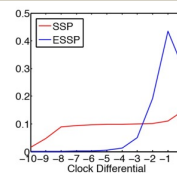
100

BAP Data Parallel: (E)SSP Probability Bound

[Dai et al., 2015]

Let **real staleness observed by system** be γ_t

Let its mean, variance be $\mu_\gamma = \mathbb{E}[\gamma_t]$, $\sigma_\gamma = \text{var}(\gamma_t)$



Theorem: Given L-Lipschitz objective f_t and stepsize h_t ,

$$P \left[\underbrace{\frac{R[X]}{T}}_{\text{Gap between current estimate and optimum}} - \frac{1}{\sqrt{T}} \left(\eta L^2 + \underbrace{\frac{F^2}{\eta}}_{\text{Penalty due to high avg. staleness } \mu_{stale}} + 2\eta L^2 \underbrace{\mu_\gamma}_{\text{Penalty due to high staleness var. } \sigma_{stale}} \right) \geq \tau \right] \leq \exp \left\{ \frac{-T\tau^2}{2\bar{\eta}_T \sigma_\gamma + \frac{2}{3}\eta L^2 (2s+1)P\tau} \right\}$$

$$R[X] := \sum_{t=1}^T f_t(\tilde{x}_t) - f(x^*) \quad \bar{\eta}_T = \frac{\eta^2 L^4 (\ln T + 1)}{T} = o(T)$$

Explanation: the (E)SSP distance between true optima and current estimate decreases exponentially with more iterations. **Lower staleness mean, variance $\mu_\gamma, \sigma_\gamma$ improve the convergence rate.**

Take-away: controlling staleness mean μ_γ , variance σ_γ (on top of max staleness s) is needed for faster ML convergence, which ESSP does.



10

101

BAP Data Parallel: (E)SSP Variance Bound

[Dai et al., 2015]

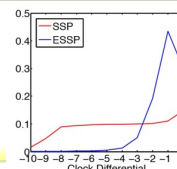
Theorem: the variance in the (E)SSP estimate is

$$\text{Var}_{t+1} = \text{Var}_t - 2\eta_t \text{cov}(\mathbf{x}_t, \mathbb{E}^{\Delta_t}[\mathbf{g}_t]) + \mathcal{O}(\eta_t \xi_t) + \mathcal{O}(\eta_t^2 \rho_t^2) + \mathcal{O}_{\gamma_t}^*$$

where

$$\text{cov}(\mathbf{a}, \mathbf{b}) := \mathbb{E}[\mathbf{a}^T \mathbf{b}] - \mathbb{E}[\mathbf{a}^T] \mathbb{E}[\mathbf{b}]$$

and $\mathcal{O}_{\gamma_t}^*$ represents 5th order or higher terms in γ_t



Explanation: The variance in the (E)SSP parameter estimate monotonically decreases when close to an optimum.

Lower (E)SSP staleness $\gamma_t \Rightarrow$ Lower variance in parameter $\Rightarrow$ Less oscillation in parameter $\Rightarrow$ More confidence in estimate quality and stopping criterion.

Take-away: Lower average staleness (via ESSP) not only improves convergence speed, but also yields better parameter estimates

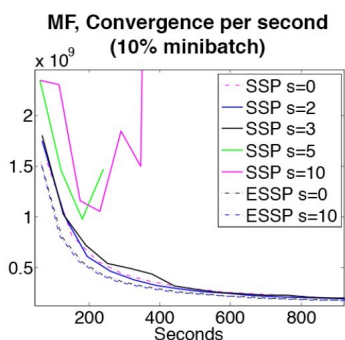


102

102

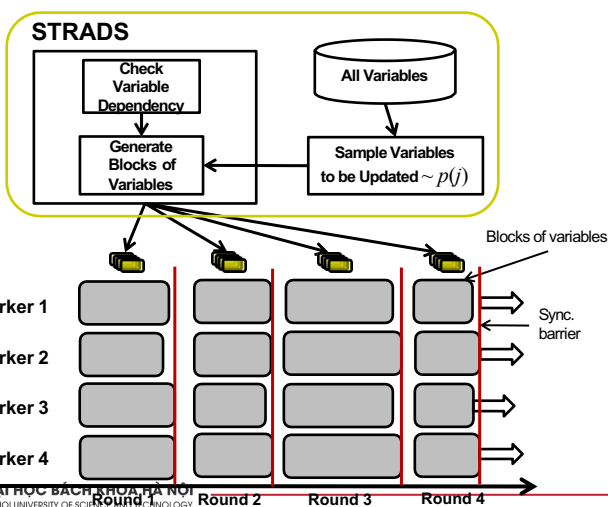
ESSP vs SSP: Increased stability helps empirical performance

- Low-staleness SSP and ESSP converge equally well
- But at higher staleness, ESSP is more stable than SSP
 - ESSP communicates updates early, whereas SSP waits until the last second
 - ESSP better suited to real-world clusters, with straggler and multi-user issues



Scheduled Model Parallel: Dynamic/Block Scheduling

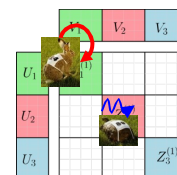
[Lee et al. 2014, Kumar et al. 2014]



• Priority Scheduling

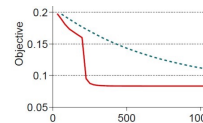
$$\{j\} \leftarrow \{j^{(t-1)}\} \oplus \mathcal{H}$$

• Block scheduling



Scheduled Model Parallel: Dynamic Scheduling Expectation Bound

[Lee et al. 2014]



- **Goal:** solve sparse regression problem $\min_{\mathbf{y}} \|\mathbf{y} - \mathbf{X}(\mathbf{J} \mathbf{k}_2^2 + \dots)\|_2$
- Via coordinate descent over "SAP blocks" $\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(B)}$
 - $\mathbf{X}^{(b)}$ are the data columns (features) in block (b)
- P parallel workers, M -dimensional data
- ρ = Spectral Radius[BlockDiag[[$\mathbf{X}^{(1)\top} \mathbf{X}^{(1)}$, ..., $\mathbf{X}^{(B)\top} \mathbf{X}^{(B)}$]]; this block-diagonal matrix quantifies the maximum level of correlation (and hence problem difficulty) within all the SAP blocks $\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(B)}$
- **SAP converges according to**

$$\mathbb{E} \left[\underbrace{f(\mathbf{X}^{(t)}) - f(\mathbf{X}^*)}_{\text{Gap between current parameter estimate and optimum}} \right] \leq \frac{\overbrace{\mathcal{O}(M)}^{\text{SAP explicitly minimizes } \rho, \text{ ensuring as close to } 1/P \text{ convergence as possible}}}{P - \frac{\mathcal{O}(P^2 \rho)}{M}} \frac{1}{t} = \mathcal{O} \left(\frac{1}{Pt} \right)$$

- **Take-away:** SAP minimizes ρ by searching for feature subsets $\mathbf{X}^{(1)}, \mathbf{X}^{(2)}, \dots, \mathbf{X}^{(B)}$ without cross-correlation $\Rightarrow$ as close to P -fold speedup as possible



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

105

105

Scheduled Model Parallel: Dynamic Scheduling Expectation Bound is near-ideal

[Xing et al. 2015]

Let $S^{ideal}()$ be an ideal model-parallel schedule

Let $\mathbf{y}^{(t)}_{idea}$ be the parameter trajectory due to ideal scheduling

Let $\mathbf{y}^{(t)}_{dyn}$ be the parameter trajectory due to SAP scheduling

Theorem: After t iterations, we have

$$\mathbb{E} [\|\mathbf{y}^{(t)}_{idea} - \mathbf{y}^{(t)}_{dyn}\|] \leq \frac{2M}{(t+1)^2} \mathbf{X}^\top \mathbf{X}$$

Explanation: Under dynamic scheduling, algorithmic progress is nearly as good as ideal model-parallelism.

Intuitively, this is because both ideal and SAP model-parallelism minimize the parameter dependencies between parallel workers.



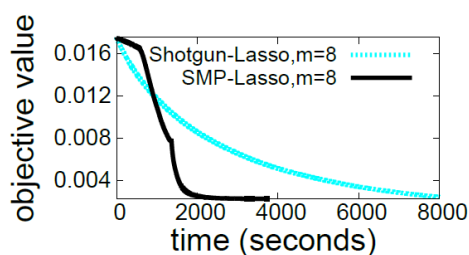
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

106

106

Scheduled Model Parallel: Dynamic Scheduling Empirical Performance

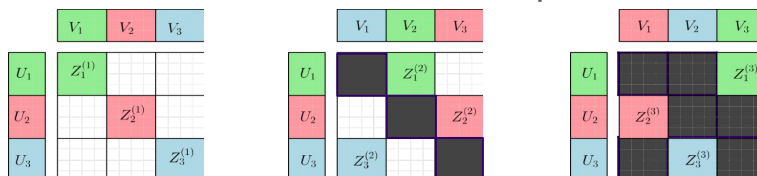
- Dynamic Scheduling for Lasso regression (SMP-Lasso):
almost-ideal convergence rate, much faster than random scheduling (Shotgun-Lasso)



Scheduled Data+Model Parallel: Block-based Scheduling (with load balancing)

[Kumar et al. 2014]

Partition data & model into $d \times d$ blocks
Run different-colored blocks in parallel



Blocks with less data/para or experience less straggling run more iterations
Automatic load-balancing + better convergence

Scheduled Data+Model Parallel: Block-based Scheduling Variance Bound 1

[Kumar et al. 2014]

- Variance between iterations S_n+1 and S_n is:

$$\begin{aligned} \text{Var}(\Psi_{S_n+1}) &= \text{Var}(\Psi_{S_n}) - 2\eta_{S_n} \sum_{i=1}^w n_i \Omega_0^i \text{Var}(\psi_{S_n}^i) \\ &\quad - 2\eta_{S_n} \sum_{i=1}^w n_i \Omega_0^i \text{CoVar}(\psi_{S_n}^i, \bar{\delta}_{S_n}^i) + \eta_{S_n}^2 \sum_{i=1}^w n_i \Omega_1^i + \mathcal{O}(\Delta_{S_n}) \end{aligned}$$

- Explanation:
 - higher order terms (red) are negligible
 - => parameter variance decreases every iteration
- Every iteration, the parameter estimates become more stable

Scheduled Data+Model Parallel: Block-based Scheduling Variance Bound 2

[Kumar et al. 2014]

- Intra-block variance: Within blocks, suppose we update the parameters ψ using n_i data points. Then, variance of ψ after those n_i updates is:

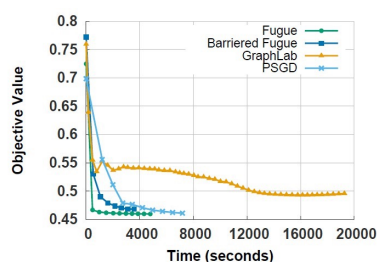
$$\begin{aligned} \text{Var}(\psi^{t+n_i}) &= \text{Var}(\psi^t) - 2\eta_t n_i \Omega_0 (\text{Var}(\psi^t)) \\ &\quad - 2\eta_t n_i \Omega_0 \text{CoVar}(\psi^t, \bar{\delta}_t) + \eta_t^2 n_i \Omega_1 \\ &\quad + \underbrace{\mathcal{O}(\eta_t^2 \rho_t) + \mathcal{O}(\eta_t \rho_t^2) + \mathcal{O}(\eta_t^3) + \mathcal{O}(\eta_t^2 \rho_t^2)}_{\Delta_t} \end{aligned}$$

- Explanation:
 - Higher order terms (red) are negligible
 - => doing more updates within each block decreases parameter variance, leading to more stable convergence

Using batching by doing extra updates is effective

Scheduled Data+Model Parallel: Block-Scheduling Empirical Performance

- Slow-worker Agnostic Block-Scheduling (Fugue) faster than:
 - Embarrassingly Parallel SGD (PSGD)
 - Non slow-worker Agnostic Block-Scheduling (Barriered Fugue)
- Slow-worker Agnostic Block-Scheduling converges to a better optimum than asynchronous GraphLab
 - Reason: more stable convergence due to block-scheduling
- Task: Imagenet Dictionary Learning
 - 630k images, 1k features



BAP Model-Parallel Guarantees

- Model-parallel under synchronous setting:
 - Dynamic scheduling
 - Slow-worker block-based scheduling
- Synchronous slow-worker problem solved by:
 - Load balancing (for dynamic scheduling)
 - Allow additional iters while waiting for other workers (slow-worker scheduling)
- Work in progress: theoretical guarantees for bounded-async model-parallel execution
 - Intuition: model-parallel sub-problems are nearly independent (thanks to scheduling)
 - Perhaps better per-iteration convergence than bounded-async data-parallel learning?

Summary

- ML Programs different from Operational Programs
 - Error tolerant – allows bounded asynchronous execution
 - Dependency structures – will slow down convergence if ignored
 - Non-uniform convergence – can allocate resources more efficiently
- Distributed Systems are Challenging
 - Uneven machine performance – must deal with slow workers/stragglers
 - Communication bottlenecks – due to iterative algo updates on Big Models
- Data, Model-parallelism to understand ML algorithms, and build distributed ML systems
 - How to distribute ML computation?
 - How to bridge ML computation and communication?
 - How to perform ML communication?
- Theory to understand why/how distributed ML systems work